

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN ĐIỆN TỬ

BỘ MÔN ĐIỆN TỬ - THÔNG TIN



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

**NGHIÊN CỨU VÀ TRIỂN KHAI HỆ THỐNG
ADAS TỰ ĐỘNG HIỆU CHUẨN TÍCH HỢP
CAN-BUS VÀ DỰ ĐOÁN HÀNH VI LÁI XE
TRÊN NỀN TẢNG EDGE AI**

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

GVHD : TS. Huỳnh Thế Thiện

SVTH : Nguyễn Thị Thủy

MSSV : 22139069

Khóa : 2022

TP. Hồ Chí Minh – 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ - THÔNG TIN



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

**NGHIÊN CỨU VÀ TRIỂN KHAI HỆ THỐNG
ADAS TỰ ĐỘNG HIỆU CHUẨN TÍCH HỢP
CAN-BUS VÀ DỰ ĐOÁN HÀNH VI LÁI XE
TRÊN NỀN TẢNG EDGE AI**

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

GVHD : TS. Huỳnh Thế Thiện

SVTH : Nguyễn Thị Thủy

MSSV : 22139069

Khóa : 2022

TP. Hồ Chí Minh – 06/2026

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: NGHIÊN CỨU VÀ TRIỂN KHAI HỆ THỐNG ADAS TỰ ĐỘNG HIỆU CHUẨN TÍCH
HỢP CAN-BUS VÀ DỰ ĐOÁN HÀNH VI LÁI XE TRÊN NỀN TẢNG EDGE AI
Sinh viên: Nguyễn Thị Thủy MSSV: 22139069
Hướng dẫn: TS. Huỳnh Thế Thiện

Nhận xét bao gồm các nội dung sau đây:

- Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:** Đề tài đặt vấn đề rất rõ ràng, bám sát nhu cầu thực tiễn về hệ thống giao thông thông minh. Việc giải quyết bài toán bằng cách triển khai các mô hình học sâu có kiến trúc tối ưu tham số (parameter-efficient) như YOLOv11n trên thiết bị biên (Edge AI) thay vì phụ thuộc xử lý đám mây thể hiện tính mới, tính cấp thiết và khả năng ứng dụng thực tiễn rất cao.
- Phương pháp thực hiện/ phân tích/ thiết kế:** Sinh viên đã vận dụng phương pháp nghiên cứu tin cậy, kết hợp chặt chẽ giữa cơ sở lý thuyết toán học và thực nghiệm. Thiết kế kiến trúc đa mô hình (Multi-Model Pipeline) tích hợp định dạng lượng tử hóa INT8 cho thấy tư duy kỹ thuật xuất sắc, phù hợp tối đa với các hệ thống nhúng IoT tại biên có tài nguyên phần cứng hạn chế.
- Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:** Kết quả đạt được hoàn toàn đáp ứng mục tiêu đề tài. Sinh viên đã xây dựng thành công khung xử lý đồng bộ dữ liệu từ hình ảnh, mạng CAN-Bus và IMU với độ trễ thấp. Các kịch bản kiểm thử (SiL, HiL, thực địa) được thiết kế và đánh giá tương đối chặt chẽ, chứng minh độ tin cậy vững chắc của thuật toán Auto-Calibration và logic tính toán chỉ số TTC.
- Kết luận và đề xuất:** Kết luận tổng kết đầy đủ và bám sát kết quả thực nghiệm. Các đề xuất mở rộng hướng phát triển như tích hợp V2X và thu nhỏ thiết kế phần cứng SoC là khả thi, mang tính cải tiến và có sức thuyết phục cao.
- Hình thức trình bày và bố cục báo cáo:** Báo cáo được trình bày khoa học, văn phong học thuật mạch lạc, nhất quán. Cấu trúc các chương hợp lý. Sinh viên đã nghiêm túc tiếp thu các góp ý sửa đổi, văn bản được trau chuốt cẩn thận, các công thức toán học và biểu đồ đánh giá hiệu năng được trình bày chuẩn xác, thể hiện năng lực chuyên môn tốt và sự chính chu.
- Kỹ năng chuyên nghiệp và tính sáng tạo:** Sinh viên thể hiện tính chủ động, tư duy làm việc độc lập và khả năng giải quyết các vấn đề kỹ thuật phức tạp (tối ưu bộ nhớ, xử lý đa luồng) một cách đầy sáng tạo. Kỹ năng trình bày và phản hồi chuyên môn tốt.
- Tài liệu trích dẫn:** Tài liệu tham khảo tương đối đa dạng, có tính cập nhật cao. Việc trích dẫn được thực hiện trung thực, tuân thủ đúng định dạng quy chuẩn IEEE/APA, thể hiện nền tảng nghiên cứu lý thuyết vững chắc.
- Đánh giá về sự trùng lặp của đề tài:** Đề tài ít có sự trùng lặp với các nghiên cứu đã công bố. Mã nguồn tối ưu, thuật toán hợp nhất cảm biến và thiết kế phần cứng là kết quả nghiên cứu độc lập của sinh viên.
- Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa:** Sinh viên đã khắc phục các sai sót về lỗi đánh số, hiệu chỉnh lại biểu đồ so sánh FPS công bằng và hoàn thiện hệ thống công thức toán học như yêu cầu. Hiện tại báo cáo không còn thiếu sót lớn, chất lượng đủ tốt để có thể phát triển thành bài báo hội nghị chuyên ngành.
- Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên:** Thái độ nghiên cứu nghiêm túc, chịu khó tìm tòi và bắt kịp các công nghệ kỹ thuật Edge AI mới nhất. Luôn tuân thủ đúng tiến độ đề ra.

Đề nghị của giảng viên hướng dẫn:

Báo cáo hoàn toàn đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép bảo vệ khóa luận tốt nghiệp trước Hội đồng.

Tp. HCM, ngày 23 tháng 06 năm 2026

Người nhận xét

(Ký và ghi rõ họ tên)

TS. Huỳnh Thế Thiện

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ và tên sinh viên: Nguyễn Thị Thủy

MSSV: 22139069

1. Tên đề tài: Nghiên cứu và triển khai hệ thống ADAS tự động hiệu chuẩn tích hợp CAN-Bus và dự đoán hành vi lái xe trên nền tảng Edge AI

2. Các số liệu, tài liệu ban đầu:

- Lý thuyết về ADAS, Edge AI, mạng CAN-Bus, thuật toán học sâu và bộ lọc dữ liệu.
- Các bộ dữ liệu dataset giao thông thực tế và chuỗi thời gian hành vi lái xe.
- Phác thảo kiến trúc phần mềm đa luồng và mô hình hệ thống tổng thể.

3. Nội dung thực hiện đề tài:

- Thiết kế và thi công mô hình phần cứng tổng thể.
- Lập trình phần mềm đa luồng: xử lý thị giác máy tính, hiệu chuẩn tự động, tính TTC và phân tích hành vi.
- Kiểm thử hệ thống qua các kịch bản SiL, HiL, môi trường thực tế và viết báo cáo.

4. Sản phẩm:

- Mô hình phần cứng ADAS Edge AI kèm mạch giả lập tín hiệu và giao diện HMI.
- Mã nguồn phần mềm giám sát, cảnh báo an toàn và hộp đen lưu trữ dữ liệu.
- Quyển báo cáo đồ án tốt nghiệp.

TRƯỞNG NGÀNH

GIẢNG VIÊN HƯỚNG DẪN

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Ban Giám hiệu, cùng toàn thể Quý thầy cô Khoa Điện - Điện tử tại Trường Đại học Công nghệ Kỹ thuật TP.HCM đã tận tình giảng dạy, truyền đạt những kiến thức chuyên môn quý báu trong suốt quá trình em học tập và rèn luyện tại trường. Những kiến thức nền tảng này chính là hành trang vững chắc để em có thể nghiên cứu và hoàn thành đồ án tốt nghiệp này.

Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc đến Thầy Huỳnh Thế Thiện, người đã trực tiếp hướng dẫn, định hướng và dành nhiều thời gian, tâm huyết để hỗ trợ em trong suốt thời gian thực hiện đề tài. Những lời khuyên chuyên môn, sự động viên và định hướng khoa học của Thầy là yếu tố quyết định giúp em vượt qua những khó khăn trong quá trình nghiên cứu, lắp ráp phần cứng và phát triển thuật toán.

Em cũng xin gửi lời cảm ơn đến gia đình, bạn bè và những người thân đã luôn động viên, tạo điều kiện tốt nhất về tinh thần lẫn vật chất để em có thể yên tâm học tập và hoàn thành tốt nhiệm vụ nghiên cứu.

Mặc dù đã nỗ lực hết mình, nhưng do hạn chế về mặt thời gian và kinh nghiệm thực tiễn, đồ án chắc chắn không tránh khỏi những thiếu sót nhất định. Em rất mong nhận được sự góp ý, chỉ bảo từ Quý thầy cô trong Hội đồng đánh giá để đề tài được hoàn thiện hơn và làm tiền đề cho những nghiên cứu sâu rộng hơn trong tương lai.

Em xin chân thành cảm ơn!

LỜI CAM ĐOAN

Em tên là: Nguyễn Thị Thủy – MSSV: 22139069

Em xin cam đoan đề tài đồ án tốt nghiệp "**Nghiên cứu và triển khai hệ thống ADAS tự động hiệu chuẩn tích hợp CAN-Bus và dự đoán hành vi lái xe trên nền tảng Edge AI**" là công trình nghiên cứu của riêng cá nhân em, được thực hiện dưới sự hướng dẫn khoa học của Thầy Huỳnh Thế Thiện.

Toàn bộ các nội dung lý thuyết, phương pháp nghiên cứu, mã nguồn phần mềm, thiết kế phần cứng và kết quả thực nghiệm trình bày trong báo cáo đều mang tính trung thực và chưa từng được công bố tại bất kỳ công trình nào khác. Các số liệu, hình ảnh, tài liệu tham khảo sử dụng trong đồ án đều được trích dẫn nguồn gốc rõ ràng, minh bạch theo đúng quy định.

Nếu có bất kỳ sự gian lận hay sao chép trái phép nào, em xin hoàn toàn chịu trách nhiệm trước Hội đồng bảo vệ đồ án và Ban Giám hiệu Nhà trường.

TP. Hồ Chí Minh, ngày 24 tháng 06 năm 2026

Sinh viên thực hiện



Nguyễn Thị Thủy

TÓM TẮT

Sự gia tăng nhanh chóng của các phương tiện giao thông đặt ra những thách thức lớn về an toàn đường bộ, đòi hỏi sự can thiệp của các công nghệ hỗ trợ lái xe thông minh nhằm giảm thiểu tai nạn do lỗi chủ quan của con người. Đồ án này tập trung nghiên cứu, thiết kế và hiện thực hóa một Hệ thống Hỗ trợ Lái xe Nâng cao (ADAS) hoạt động độc lập trên nền tảng điện toán tại biên (Edge AI), sử dụng máy tính nhúng Raspberry Pi 5.

Hệ thống tích hợp công nghệ thị giác máy tính và trí tuệ nhân tạo thông qua mô hình YOLOv11n (đã được lượng tử hóa định dạng INT8 nhằm tối ưu tài nguyên) kết hợp với các thuật toán xử lý ảnh OpenCV để nhận diện vật thể, phát hiện ranh giới làn đường. Điểm đột phá của đề tài là việc áp dụng phương pháp hợp nhất đa cảm biến (Sensor Fusion): kết hợp dữ liệu hình ảnh, động lực học từ mạng CAN-Bus nội bộ và bù trừ rung lắc thông qua bộ lọc Kalman từ cảm biến quán tính IMU. Các thông số này đóng vai trò cốt lõi trong việc hiệu chuẩn camera tự động (Auto-Calibration) và tính toán chỉ số Thời gian đến va chạm TTC với độ trễ cực thấp. Ngoài ra, việc sử dụng mạng nơ-ron hồi quy LSTM giúp phân tích và dự đoán thành công các hành vi lái xe nguy hiểm.

Kết quả thử nghiệm qua các kịch bản SiL,HiL và môi trường giao thông thực tế cho thấy hệ thống hoạt động ổn định với tốc độ xử lý 22 - 28 FPS. Các tính năng cốt lõi như cảnh báo tiền va chạm (FCW), cảnh báo chệch làn thông minh (Smart LDW), dẫn đường động, và hộp đen viễn trắc đều phản hồi chính xác, minh chứng cho tính khả thi và tiềm năng ứng dụng thực tiễn của giải pháp trong lĩnh vực xe tự hành và giao thông thông minh.

MỤC LỤC

LỜI CẢM ƠN	i
LỜI CAM ĐOAN	ii
TÓM TẮT	iii
MỤC LỤC	iv
DANH MỤC HÌNH ẢNH	viii
DANH MỤC BẢNG BIỂU	xi
DANH MỤC CÁC TỪ VIẾT TẮT	xii
CHƯƠNG 1 TỔNG QUAN	1
1.1 GIỚI THIỆU	1
1.2 MỤC TIÊU NGHIÊN CỨU	2
1.3 PHƯƠNG PHÁP NGHIÊN CỨU	3
1.3.1 Phương pháp nghiên cứu lý thuyết	4
1.3.2 Phương pháp thực nghiệm và phát triển hệ thống	4
1.3.3 Phương pháp phân tích, đánh giá và kiểm thử	5
1.4 GIỚI HẠN NGHIÊN CỨU	6
1.5 BỐ CỤC	7
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT	8
2.1. TỔNG QUAN VỀ HỆ THỐNG ADAS	8
2.1.1. Khái niệm và phân cấp tự động hóa theo tiêu chuẩn SAE	8
2.1.2. Kiến trúc cơ bản của một node ADAS tại biên (Edge Computing)	9

2.2. MẠNG TRUYỀN THÔNG XE HƠI (CAN-BUS)	11
2.2.1. Nguyên lý hoạt động và kiến trúc lớp vật lý của CAN	11
2.2.2. Cấu trúc khung truyền dữ liệu và cơ chế phân xử quyền ưu tiên	12
2.3. CƠ SỞ LÝ THUYẾT THỊ GIÁC MÁY TÍNH VÀ TỰ ĐỘNG HIỆU CHUẨN	13
2.3.1. Mô hình Camera lỗ kim (Pinhole Camera Model)	13
2.3.2. Lý thuyết Tự động hiệu chuẩn (Auto-Calibration)	15
2.4. HỌC SÂU TRONG NHẬN DIỆN VÀ DỰ ĐOÁN HÀNH VI	16
2.4.1. Mô hình nhận diện YOLOv11	16
2.4.2. Mạng nơ-ron hồi quy LSTM	18
2.5. LÝ THUYẾT XỬ LÝ TÍN HIỆU VÀ ĐÁNH GIÁ AN TOÀN	20
2.5.1. Lọc nhiễu cảm biến bằng Kalman Filter	20
2.5.2. Chỉ số TTC	22
2.6. TỐI ƯU HÓA MÔ HÌNH TRÊN THIẾT BỊ EDGE AI	23
2.6.1. Nguyên lý Lượng tử hóa (Quantization INT8)	23
2.6.2. Tăng tốc phần cứng sử dụng TensorRT/OpenVINO	24
CHƯƠNG 3 THIẾT KẾ HỆ THỐNG	26
3.1. YÊU CẦU CỦA HỆ THỐNG	26
3.2. SƠ ĐỒ ĐẶC TẢ HỆ THỐNG	27
3.3. SƠ ĐỒ KHỐI HỆ THỐNG	28
3.4. THIẾT KẾ CHI TIẾT CÁC KHỐI PHẦN CỨNG	31
3.4.1. Khối xử lý trung tâm	31
3.4.2. Khối Thị giác Máy tính	32

3.4.3. Khối Giao tiếp CAN-Bus & Giả lập phương tiện	32
3.5. THIẾT KẾ PHẦN MỀM	37
3.5.1. Biện luận lựa chọn kiến trúc Đa mô hình (Multi-Model Pipeline)	38
3.5.2. Thiết kế mô-đun Thị giác máy tính (Computer Vision Pipeline)	42
3.5.3. Tích hợp Trí tuệ nhân tạo (AI Module)	43
3.5.4. Tích hợp IMU và Tự động hiệu chuẩn (Auto-Calibration)	45
3.5.5. Xử lý CAN-Bus và Logic Cảnh báo	45
CHƯƠNG 4 KẾT QUẢ VÀ THẢO LUẬN	48
4.1. HIỆU SUẤT NHẬN DIỆN ĐỐI TƯỢNG VÀ PHÁT HIỆN LÀN ĐƯỜNG	48
4.2. ĐÁNH GIÁ HIỆU NĂNG XỬ LÝ TRÊN THIẾT BỊ NHÚNG RASPBERRY PI 5	50
4.3. ĐÁNH GIÁ LOGIC CẢNH BÁO VA CHẠM (TTC) VÀ HỢP NHẤT CẢM BIẾN	55
4.4. ĐÁNH GIÁ THỰC NGHIỆM CÁC TÍNH NĂNG CỐT LÕI QUẢ KỊCH BẢN KIỂM THỬ	57
4.4.1. Phân tích Đa làn và Cảnh báo Chệch làn Thông minh (Smart LDW)	57
4.4.2. Cảnh báo Tiền va chạm và Khẩn cấp (FCW & Critical Warning)	59
4.4.3. Hợp nhất Cảm biến và Tự động Hiệu chuẩn (Sensor Fusion & Auto-Calibration)	61
4.4.4. Đánh giá Hành vi Lái xe (Driver Behavior - LSTM)	62

4.4.5. Dẫn đường Động và Giám sát Chệch hướng (Dynamic Navigation & Rerouting)	63
4.4.6. Hộp đen Dữ liệu và Đo lường Viễn trắc (Blackbox & Telemetry)	64
4.5. ĐÁNH GIÁ THỰC NGHIỆM QUA CÁC KỊCH BẢN VẬN HÀNH	65
4.5.1. Kịch bản 1: Kiểm thử Software-in-the-Loop (SiL)	65
4.5.2. Kịch bản 2: Mô phỏng Hardware-in-the-Loop (HiL) tại trạm thử nghiệm tĩnh	66
4.5.3. Kịch bản 3: Thực nghiệm động lực học trên môi trường giao thông thực tế	68
CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	71
5.1. KẾT LUẬN	71
5.1.1 Kết quả đạt được	71
5.1.2 Hạn chế	72
5.2. HƯỚNG PHÁT TRIỂN	73
TÀI LIỆU THAM KHẢO	76

DANH MỤC HÌNH ẢNH

Hình 1.1: Thống kê tai nạn giao thông đường bộ tại Việt Nam giai đoạn 2020-2025	1
Hình 2.1: Phân cấp các cấp độ tự động hóa lái xe theo tiêu chuẩn SAE J3016	8
Hình 2.2: Kiến trúc luồng xử lý dữ liệu đa tầng của một node ADAS tại biên	10
Hình 2.4: Cấu trúc các trường dữ liệu chi tiết của hai loại khung truyền dữ liệu	12
Hình 2.5: Phép chiếu phối cảnh từ hệ tọa độ 3D thế giới thực sang mặt phẳng ảnh 2D thông qua tâm quang học	14
Hình 2.6: Mô hình xác định điểm tụ từ các đường lằn đường song song để suy xuất góc Pitch và Yaw của camera	15
Hình 2.7: Sơ đồ cấu trúc các phân hệ Backbone (khối C3k2, C2PSA), Neck và Decoupled Head của mạng YOLOv11	17
Hình 2.8: Kiến trúc cấu tạo nội tại của một ô nhớ LSTM với các cổng logic Forget gate, Input gate và Output gate	19
Hình 2.9: Vòng lặp các bước Dự đoán (Predict) và Cập nhật (Update) hiệu chỉnh trạng thái của bộ lọc Kalman	21
Hình 2.10: Mô hình hình học động lực học xác định khoảng cách dọc Z và tính toán chỉ số Time-to-Collision (TTC) từ camera monocular	22
Hình 2.11: Đồ thị mô tả ánh xạ dải phân bố trọng số từ định dạng số thực FP32 sang số nguyên khép kín INT8	24
Hình 3.1: Sơ đồ đặc tả tổng thể hệ thống ADAS Edge AI	28
Hình 3.2: Sơ đồ khối chức năng và luồng hoạt động của hệ thống	30
Hình 3.3: Sơ đồ nguyên lý kết nối toàn bộ hệ thống phần cứng	36
Hình 3.4: Thiết kế bố trí các module phần cứng thực tế	37

Hình 3.5: Lưu đồ thuật toán tổng thể của hệ thống (Pipeline Đa luồng)	38
Hình 3.6: Lưu đồ chi tiết module xử lý thị giác và Lane Detection	43
Hình 3.7: Kết quả nhận diện đối tượng và vẽ hộp giới hạn	44
Hình 3.8: Lưu đồ module tính toán TTC và Logic Cảnh báo	46
Hình 3.9: Giao diện Web Dashboard giám sát hệ thống	47
Hình 4.1: Ma trận nhầm lẫn (Confusion Matrix) đánh giá độ chính xác nhận diện của YOLOv11 đối với các lớp đối tượng giao thông	49
Hình 4.2: Trực quan hóa kết quả nhận diện hộp giới hạn (Bounding Box) và phát hiện làn đường (Lane Overlay) trong điều kiện thực tế	50
Hình 4.3: Biểu đồ so sánh tốc độ khung hình (FPS) giữa các phương pháp tối ưu hóa mô hình trên thiết bị biên	51
Hình 4.4: So sánh hiệu năng tốc độ xử lý (FPS) thực nghiệm của hệ thống đa mô hình phân tán với các kiến trúc công nghệ thay thế trên Raspberry Pi 5	53
Hình 4.5: Đồ thị thể hiện sự thay đổi của chỉ số TTC theo thời gian thực và các mốc kích hoạt cảnh báo tương ứng	56
Hình 4.6: Giao diện HMI trong kịch bản chệch làn nguy hiểm	58
Hình 4.7: Giao diện HMI trong kịch bản chuyển làn có chủ đích	59
Hình 4.8: Mức độ phản hồi của hệ thống đối với cảnh báo tiền va chạm ($TTC < 2.5s$) và khẩn cấp (Khoảng cách $< 1m$)	60
Hình 4.9: Trạng thái bám làn ổn định của hệ thống nhờ thuật toán Auto-Calibration khi camera bị tác động góc nghiêng	61
Hình 4.10: Hệ thống nhận diện và cập nhật trạng thái hành vi lái xe nguy hiểm theo thời gian thực	62

Hình 4.11: Giao diện bản đồ trong quá trình dẫn đường động và phát hiện chệch hướng	63
Hình 4.12: Dữ liệu hộp đen ghi nhận và khôi phục thành công các mốc cảnh báo sự kiện trên bản đồ	64
Hình 4.13: Giao diện phần mềm và các thông số debug trong quá trình kiểm thử Software-in-the-Loop trên máy tính	66
Hình 4.14: Hình ảnh minh họa quá trình thử nghiệm kết hợp phần cứng (Hardware-in-the-Loop) với màn hình giả lập	67
Hình 4.15: Hình ảnh gá lắp hệ thống và giao diện hiển thị cảnh báo trực tiếp trong quá trình thử nghiệm trên đường thực tế	69
Hình 4.16: Mô hình thử nghiệm chạy thực tế buổi tối	70
Hình 4.17: Mô hình thử nghiệm chạy thực tế buổi sáng	70

DANH MỤC BẢNG BIỂU

Bảng 3.1: Thông số kỹ thuật cơ bản của Raspberry Pi 5	31
Bảng 3.2: Thông số kỹ thuật của Camera Module 3 Wide	32
Bảng 3.3: Thông số kỹ thuật của vi điều khiển ESP32-S3	33
Bảng 3.4: Thông số kỹ thuật của module Waveshare USB-CAN-A	33
Bảng 3.5: Thông số kỹ thuật của cảm biến IMU GY-BNO055	34
Bảng 3.6: Thông số cấu hình Khối Hiển thị & Nguồn cấp	35
Bảng 3.7: Phân tích và so sánh lựa chọn mô hình AI cho các tác vụ cốt lõi	39
Bảng 4.1: So sánh hiệu năng tính toán và tiêu thụ tài nguyên trên Raspberry Pi 5	51
Bảng 5.1: Tổng hợp và So sánh các phương pháp tiếp cận ADAS	74

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Từ đầy đủ
3GPP	3rd Generation Partnership Project
ADAS	Advanced Driver Assistance Systems
ADC	Analog-to-Digital Converter
AI	Artificial Intelligence
API	Application Programming Interface
CAN	Controller Area Network
CCTV	Closed-Circuit Television
CIoU	Complete Intersection over Union
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSI	Camera Serial Interface
CSMA/CD+AMP	Carrier Sense Multiple Access with Collision Detection and Arbitration on Message Priority
CUDA	Compute Unified Device Architecture
DFL	Distribution Focal Loss
DOF	Degrees of Freedom
ECU	Electronic Control Unit
FCW	Forward Collision Warning
FOV	Field of View

FP32	32-bit Floating Point
FPS	Frames Per Second
GPS	Global Positioning System
HiL	Hardware-in-the-Loop
HMI	Human-Machine Interface
I2C	Inter-Integrated Circuit
IMU	Inertial Measurement Unit
INT8	8-bit Integer
IoT	Internet of Things
ITS	Intelligent Transportation System
LDW	Lane Departure Warning
LSTM	Long Short-Term Memory
mAP	mean Average Precision
NPU	Neural Processing Unit
OBD-II	On-Board Diagnostics II
ONNX	Open Neural Network Exchange
OSRM	Open Source Routing Machine
PANet	Path Aggregation Network
PCB	Printed Circuit Board
PWM	Pulse Width Modulation
RNN	Recurrent Neural Network

ROI	Region of Interest
SAE	Society of Automotive Engineers
SiL	Software-in-the-Loop
SIMD	Single Instruction, Multiple Data
SoC	System on Chip
SPI	Serial Peripheral Interface
SPPF	Spatial Pyramid Pooling - Fast
TTC	Time-To-Collision
UART	Universal Asynchronous Receiver-Transmitter
UFLD	Ultra Fast Lane Detection
V2X	Vehicle-to-Everything
YOLO	You Only Look Once
CIoU	Complete Intersection over Union
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSI	Camera Serial Interface
CSMA/CD+AMP	Carrier Sense Multiple Access with Collision Detection and Arbitration on Message Priority
CUDA	Compute Unified Device Architecture
DFL	Distribution Focal Loss
DOF	Degrees of Freedom
ECU	Electronic Control Unit

CHƯƠNG 1

TỔNG QUAN

1.1 GIỚI THIỆU

Trong những năm gần đây, nhờ sự nỗ lực của các cơ quan chức năng và việc áp dụng các biện pháp quản lý nghiêm ngặt, tình hình tai nạn giao thông tại Việt Nam đã ghi nhận những chuyển biến tích cực với số vụ tai nạn nhìn chung có xu hướng giảm dần qua các năm. Tuy nhiên, sự sụt giảm này không đồng nghĩa với việc môi trường giao thông đã hoàn toàn an toàn.



Hình 1.1: Thống kê tai nạn giao thông đường bộ tại Việt Nam giai đoạn 2020-2025

Theo thống kê từ Ủy ban An toàn Giao thông Quốc gia, chỉ trong 10 tháng đầu năm 2025, cả nước đã xảy ra hơn 15.251 vụ tai nạn giao thông, khiến hơn 8.500 người thiệt mạng và hơn 10.000 người bị thương. Tỷ lệ tử vong do tai nạn giao thông tại Việt Nam vẫn ở mức cao, khoảng 17,7 ca trên 100.000 dân, cao hơn mức trung bình của nhiều quốc gia trong khu vực.

Hầu hết các vụ tai nạn hiện nay xuất phát từ lỗi chủ quan của người điều khiển như thiếu tập trung, đi sai làn đường, hay phanh gấp không giữ khoảng cách an toàn. Trong khi đó, công tác giám sát giao thông phần lớn vẫn mang tính thụ động. Việc phụ thuộc vào tuần tra thủ công hoặc hệ thống camera (CCTV) chỉ có chức năng ghi hình lưu trữ khiến việc xử lý vi phạm bị chậm trễ, thiếu đi khả năng phân tích và đưa ra các cảnh báo "phòng ngừa tức thời" ngay trước khi sự cố xảy ra.

Trong bối cảnh đó, ADAS nổi lên như một giải pháp công nghệ cấp thiết. Bằng cách ứng dụng AI, thị giác máy tính và hợp nhất dữ liệu đa cảm biến, ADAS giúp phát hiện nguy cơ và cảnh báo thời gian thực, được chứng minh có thể giảm từ 30% đến 50% tỷ lệ tai nạn tùy theo tính năng được triển khai [1], [2].

Để duy trì xu hướng giảm thiểu tai nạn, việc chuyển dịch từ "giám sát thụ động" sang "cảnh báo chủ động" là một yêu cầu tất yếu. Do đó, việc nghiên cứu và triển khai hệ thống ADAS tự động hiệu chuẩn trên nền tảng thiết bị nhúng Edge AI, kết hợp phân tích video thời gian thực và bóc tách dữ liệu CAN-Bus là một hướng đi mang tính thực tiễn cao. Hệ thống không chỉ hỗ trợ người lái đưa ra quyết định an toàn tức thời mà còn góp phần hoàn thiện bức tranh về hệ thống giao thông thông minh (ITS) tại Việt Nam.

1.2 MỤC TIÊU NGHIÊN CỨU

Mục tiêu tổng quát: Xây dựng hệ thống ADAS hoàn chỉnh trên nền tảng Edge AI (Raspberry Pi 5), tích hợp xử lý video thời gian thực, tự động hiệu chuẩn camera,

đồng bộ dữ liệu CAN-Bus và dự đoán hành vi lái xe nhằm đưa ra các cảnh báo chủ động, góp phần nâng cao an toàn giao thông.

Mục tiêu cụ thể:

Thiết lập phần cứng: Cấu hình máy tính nhúng trung tâm Raspberry Pi 5 (OS Bookworm), thiết lập giao tiếp đồng bộ với Camera Module 3 Wide, cảm biến quán tính IMU GY-BNO055 và cụm vi điều khiển ESP32 giả lập mạng CAN-Bus.

Nhận diện đối tượng và làn đường: Triển khai, tối ưu hóa mô hình YOLOv11 Nano nhằm phát hiện chính xác vật thể giao thông (ô tô, xe máy, người đi bộ) kết hợp thuật toán OpenCV để nhận diện làn đường và phát hiện trạng thái chệch làn (LDW).

Giải mã dữ liệu và Tự động hiệu chuẩn: Xây dựng module bóc tách thông số vận hành từ CAN-Bus (tốc độ, góc lái, phanh, xi-nhan) đồng thời phát triển thuật toán tự động hiệu chuẩn camera động (Auto-Calibration) dựa trên điểm tụ (Vanishing Point) và bộ lọc dữ liệu IMU.

Phân tích hành vi và Cảnh báo va chạm: Huấn luyện mạng hồi quy LSTM để phân tích và dự đoán hành vi lái xe nguy hiểm; tính toán chỉ số thời gian va chạm TTC để đưa ra các cấp độ cảnh báo an toàn và thiết kế giao diện Web-based Dashboard giám sát.

Tối ưu hiệu năng và Kiểm thử: Đảm bảo toàn bộ pipeline phần mềm hoạt động ổn định ở tốc độ thời gian thực (20-28 FPS) trên thiết bị biên và đánh giá định lượng hệ thống qua các kịch bản thực nghiệm SiL, HiL và môi trường giao thông thực tế.

1.3 PHƯƠNG PHÁP NGHIÊN CỨU

Để giải quyết một cách toàn diện bài toán tích hợp đa cảm biến và tối ưu hóa các mô hình trí tuệ nhân tạo trên phần cứng nhúng, đề án áp dụng một quy trình nghiên cứu khoa học chặt chẽ. Phương pháp luận được triển khai đồng bộ qua ba giai đoạn

cốt lõi: nghiên cứu lý thuyết nền tảng, thực nghiệm thiết kế - phát triển phần mềm, và đánh giá định lượng hiệu năng hệ thống.

1.3.1 Phương pháp nghiên cứu lý thuyết

Đây là giai đoạn tạo dựng cơ sở khoa học để định hình cấu trúc hệ thống và lựa chọn giải pháp thuật toán tối ưu. Các nội dung nghiên cứu bao gồm:

Tổng quan tiêu chuẩn viễn thông và ô tô: Nghiên cứu cấu trúc phân tầng dữ liệu, định dạng khung truyền (CAN frame) và các đặc tả kỹ thuật của mạng giao tiếp CAN-Bus nội bộ theo chuẩn 3GPP và OBD-II nhằm phục vụ cho quá trình bóc tách dữ liệu vận hành phương tiện.

Khảo sát mô hình học sâu thị giác và chuỗi thời gian: Nghiên cứu nguyên lý hoạt động của các kiến trúc mạng nơ-ron tích chập và mô hình attention trong thị giác máy tính phục vụ cho tác vụ nhận diện vật thể. Đồng thời, phân tích sâu kiến trúc mạng nơ-ron hồi quy LSTM để nắm vững phương pháp xử lý chuỗi dữ liệu thời gian ứng dụng trong phân loại hành vi lái xe.

Cơ sở hình học máy tính và lọc nhiễu tín hiệu: Nghiên cứu mô hình camera lỗ kim, các phép biến đổi không gian để tính toán điểm tụ phục vụ thuật toán tự động hiệu chuẩn. Khảo sát lý thuyết bộ lọc Kalman và bộ lọc bù nhằm triệt tiêu sai số hình học và nhiễu động học từ cảm biến quán tính IMU và GPS.

1.3.2 Phương pháp thực nghiệm và phát triển hệ thống

Giai đoạn này tập trung vào việc hiện thực hóa các mô hình lý thuyết thành sản phẩm phần cứng và phần mềm chạy ổn định trên cấu hình nhúng tại biên:

Phát triển mô hình phần cứng giả lập (Hardware-in-the-loop): Tiến hành thiết kế, lắp ráp và lập trình bằng ngôn ngữ C/C++ trên cấu hình vi điều khiển ESP32-S3 (Car Simulator Node). Module này có nhiệm vụ thu thập tín hiệu analog từ các cảm biến giả lập chân ga/góc lái và tín hiệu digital từ các nút nhấn hệ

thống phanh, xi-nhan, sau đó đóng gói thành các bản tin CAN-Bus để truyền tải lên bo mạch trung tâm.

Xây dựng pipeline xử lý phần mềm đa luồng: Lập trình trên môi trường hệ điều hành Linux của Raspberry Pi 5 bằng ngôn ngữ Python. Hệ thống phần mềm được thiết kế theo kiến trúc phân tách luồng, chạy song song các tác vụ bao gồm: luồng thu nhận hình ảnh CSI từ Camera Module 3, luồng đọc dữ liệu CAN từ Serial, luồng bóc tách dữ liệu IMU/GPS và luồng thực thi các thuật toán AI.

Huấn luyện và tối ưu hóa mô hình Edge AI: Sử dụng framework PyTorch để cấu hình và huấn luyện mạng LSTM trên bộ dữ liệu hành vi lái xe thực nghiệm nhằm phân loại chính xác các trạng thái vận hành nguy hiểm. Chuyển đổi trọng số mô hình YOLOv11 Nano và cấu hình tối ưu hóa thông qua các công cụ tăng tốc phần cứng như TensorRT hoặc OpenVINO. Tiếp tục áp dụng kỹ thuật lượng tử hóa số nguyên 8-bit (Quantization INT8) nhằm ép giảm dung lượng mô hình, nâng cao tốc độ suy luận thời gian thực.

1.3.3 Phương pháp phân tích, đánh giá và kiểm thử

Sau khi tích hợp hệ thống, các phương pháp đánh giá định lượng được áp dụng để chứng minh tính hiệu quả của giải pháp:

Đánh giá độ chính xác phân đoạn và phân loại: Kiểm thử mô hình dự đoán hành vi lái xe trên tập dữ liệu kiểm tra biệt lập, tính toán toán học các chỉ số thống kê chuẩn mực bao gồm Accuracy, Precision, Recall và F1-score để lập ma trận nhầm lẫn.

Đánh giá sai số hình học của thuật toán hiệu chuẩn và đo khoảng cách: So sánh trực tiếp các thông số góc pitch, yaw thu được từ cơ chế Auto-Calibration động với các thông số đo đạc thủ công cố định. Đo đạc sai số khoảng cách tương đối tới vật thể phía trước trong các kịch bản thực nghiệm hiện trường thực tế để hiệu chỉnh thuật toán tính toán thời gian TTC.

Đánh giá hiệu năng và tài nguyên phần cứng: Đo đặc tần suất xử lý khung hình trên giây FPS và độ trễ toàn trình latency của hệ thống. Đồng thời, thực hiện ghi log và vẽ biểu đồ giám sát mức độ chiếm dụng tài nguyên lõi CPU, dung lượng bộ nhớ RAM và kiểm soát nhiệt độ vận hành của chip xử lý Broadcom trên Raspberry Pi 5 khi chạy toàn tải đa luồng.

1.4 GIỚI HẠN NGHIÊN CỨU

Mặc dù hướng đến tính ứng dụng thực tiễn cao, đề tài vẫn tồn tại một số giới hạn nhất định trong khuôn khổ nghiên cứu:

Phạm vi quan sát: Hệ thống chủ yếu tập trung xử lý hình ảnh từ camera monocular góc rộng lắp cố định phía trước kính lái, chưa tích hợp hệ thống đa camera đa hướng hay camera 360 độ.

Điều kiện môi trường: Hiệu suất của hệ thống đạt mức tối ưu nhất trong điều kiện ánh sáng ban ngày hoặc ban đêm có hệ thống đèn đường chiếu sáng. Hiệu quả nhận diện có thể bị suy giảm đáng kể trong các điều kiện thời tiết khắc nghiệt như mưa lớn, sương mù dày đặc hoặc khi bị chói sáng mạnh.

Nền tảng phần cứng: Việc triển khai và đánh giá hiệu năng hệ thống chỉ được thực hiện trên cấu hình máy tính nhúng Raspberry Pi 5 phiên bản 8GB RAM.

Dữ liệu huấn luyện: Bộ dữ liệu dùng để huấn luyện mạng LSTM dự đoán hành vi chủ yếu dựa trên dữ liệu thực nghiệm có quy mô hạn chế và một số tập dữ liệu công khai, do đó có thể chưa bao quát được toàn bộ các kịch bản lái xe phức tạp trong thực tế.

Môi trường thử nghiệm: Ở giai đoạn hiện tại, hệ thống kết hợp giữa việc nhận dữ liệu từ cụm giả lập xe (Car Simulator Node qua ESP32) và thử nghiệm hiện trường (Field Test) quy mô nhỏ, chưa được triển khai đánh giá dài hạn trên nhiều chủng loại phương tiện thực tế khác nhau.

1.5 BỐ CỤC

Nội dung của đề án được tổ chức thành 5 chương chính như sau:

CHƯƠNG 1: TỔNG QUAN trình bày tổng quan về tính cấp thiết, mục tiêu nghiên cứu, giới hạn phạm vi và bố cục tổng thể của đề tài.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT cung cấp nền tảng kiến thức về hệ thống ADAS, Edge AI, các thuật toán thị giác máy tính, mạng CAN-Bus, kỹ thuật hợp nhất cảm biến (Kalman Filter), mô hình mạng LSTM cùng các phương pháp tối ưu hóa mô hình.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG mô tả chi tiết kiến trúc phần cứng, kiến trúc phần mềm (Data Pipeline), và phương pháp thiết kế các module cốt lõi như: Auto-Calibration, nhận diện đối tượng, ước lượng khoảng cách, nhận diện làn đường, dự đoán hành vi và giao diện Dashboard.

CHƯƠNG 4: KẾT QUẢ VÀ THẢO LUẬN trình bày quá trình triển khai thực tế, thu thập dữ liệu, tối ưu hóa hệ thống và kết quả của các kịch bản thử nghiệm.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN phân tích các chỉ số đánh giá hiệu năng đạt được, chỉ ra các mặt hạn chế còn tồn đọng và đề xuất hướng phát triển mở rộng cho hệ thống trong tương lai.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1. TỔNG QUAN VỀ HỆ THỐNG ADAS

2.1.1. Khái niệm và phân cấp tự động hóa theo tiêu chuẩn SAE

Hệ thống hỗ trợ lái xe nâng cao (ADAS) là một tổ hợp các công nghệ điện tử, cảm biến và thuật toán xử lý dữ liệu được tích hợp trên phương tiện giao thông nhằm gia tăng sự an toàn và tiện nghi cho người điều khiển. Bằng cách liên tục giám sát môi trường xung quanh phương tiện và trạng thái vận hành của xe, ADAS có khả năng phát hiện các nguy cơ tiềm ẩn, đưa ra cảnh báo kịp thời, hoặc tự động can thiệp vào hệ thống điều khiển (như phanh khẩn cấp hoặc đánh lái) để ngăn ngừa tai nạn.

Để tạo ra một hệ quy chiếu chung cho sự phát triển của công nghệ xe tự hành và ADAS, Hiệp hội Kỹ sư Ô tô Quốc tế (SAE) đã ban hành tiêu chuẩn phân loại mức độ tự động hóa thành 6 cấp độ từ Level 0 đến Level 5 [15]:

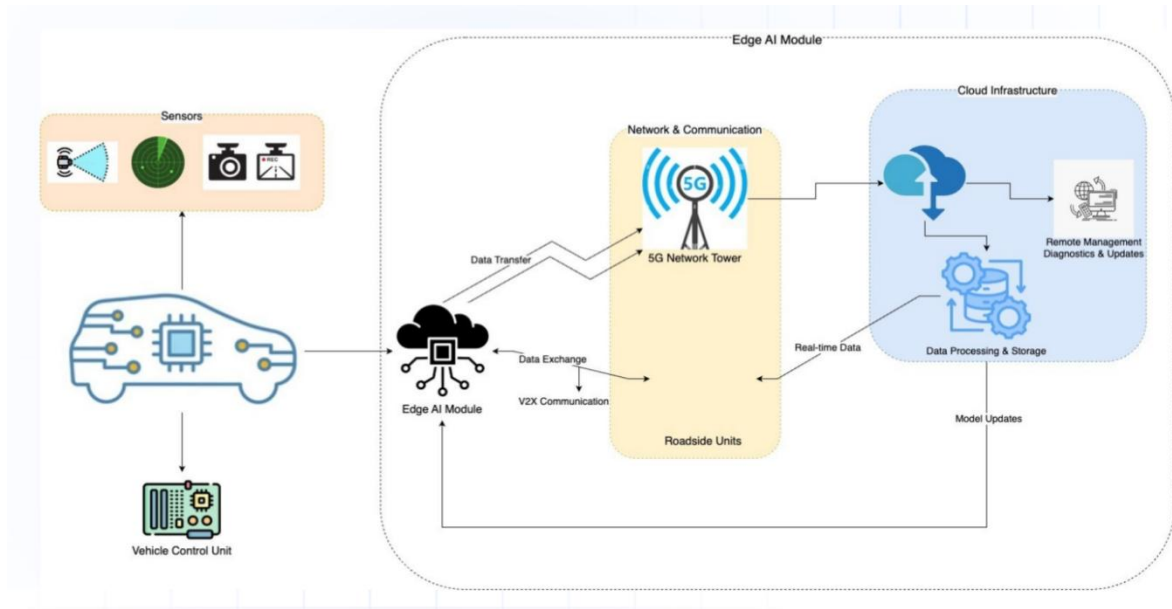


Hình 2.1: Phân cấp các cấp độ tự động hóa lái xe theo tiêu chuẩn SAE J3016

- **Cấp độ 0 (Không tự động hóa):** Người lái kiểm soát hoàn toàn mọi thao tác vận hành. Hệ thống chỉ cung cấp cảnh báo thụ động (ví dụ: cảnh báo điểm mù, cảnh báo chệch làn) mà không can thiệp vật lý.
- **Cấp độ 1 (Hỗ trợ người lái):** Hệ thống có thể hỗ trợ một chức năng điều khiển duy nhất (đánh lái HOẶC tăng tốc/phanh) trong điều kiện nhất định, ví dụ như hệ thống Kiểm soát hành trình thích ứng (ACC).
- **Cấp độ 2 (Tự động hóa một phần):** Hệ thống đồng thời kiểm soát cả việc đánh lái VÀ gia tốc/phanh. Tuy nhiên, người lái vẫn phải liên tục giám sát môi trường và sẵn sàng can thiệp.
- **Cấp độ 3 (Tự động hóa có điều kiện):** Hệ thống tự thực hiện toàn bộ tác vụ lái xe trong một số kịch bản giao thông cụ thể. Người lái không cần giám sát liên tục nhưng phải tiếp quản quyền điều khiển khi có cảnh báo yêu cầu.
- **Cấp độ 4 (Tự động hóa cao):** Phương tiện tự vận hành hoàn toàn trong các khu vực địa lý hoặc điều kiện thời tiết được định nghĩa trước. Nếu xảy ra sự cố, xe sẽ tự đưa ra phương án an toàn (fall-back) mà không cần người lái.
- **Cấp độ 5 (Tự động hóa hoàn toàn):** Hệ thống xử lý mọi tình huống giao thông tương đương người lái xe có kinh nghiệm ở bất kỳ đâu, không cần vô lăng hay bàn đạp.

2.1.2. Kiến trúc cơ bản của một node ADAS tại biên (Edge Computing)

Thay vì phụ thuộc hoàn toàn vào điện toán đám mây vốn dễ gặp rủi ro về độ trễ mạng, các hệ thống ADAS hiện đại dịch chuyển năng lực tính toán về sát nguồn phát sinh dữ liệu, gọi là điện toán tại biên (Edge Computing) [4].



Hình 2.2: Kiến trúc luồng xử lý dữ liệu đa tầng của một node ADAS tại biên

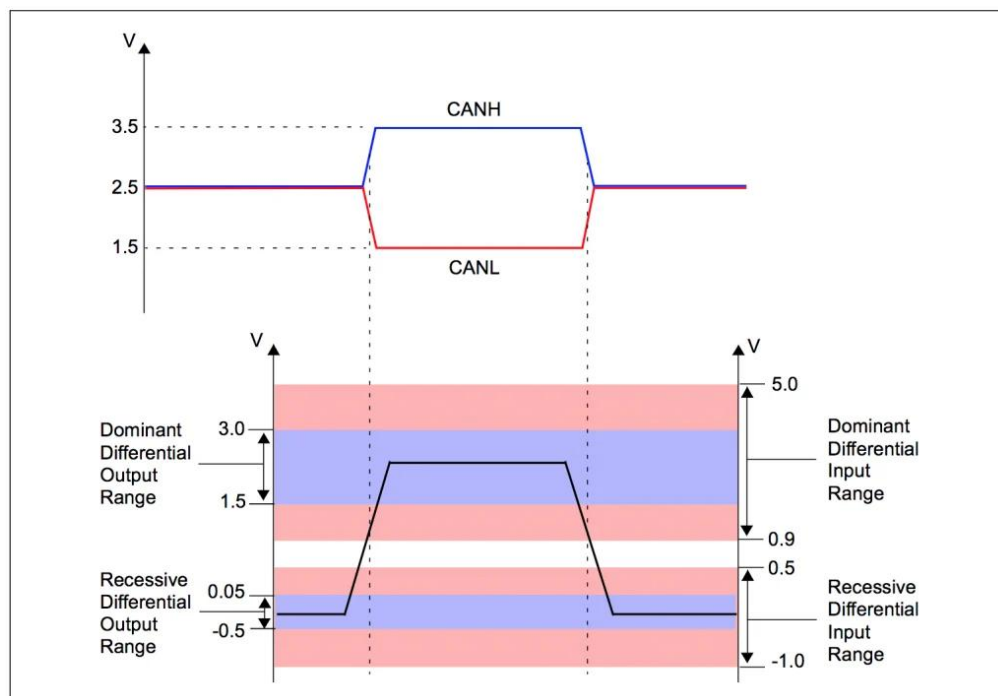
Trong đồ án này, một node ADAS tại biên hoạt động độc lập trên thiết bị nhúng (như Raspberry Pi 5 [10]) bao gồm các khối cốt lõi:

- Khối thu thập dữ liệu: Trực tiếp ghi nhận tín hiệu môi trường qua camera, IMU (gia tốc, góc nghiêng), và GPS.
- Khối giao tiếp mạng: Cầu nối với mạng nội bộ của xe để trích xuất các thông số vận hành theo thời gian thực.
- Khối xử lý trung tâm: Thực thi các thuật toán thị giác máy tính, hợp nhất dữ liệu đa cảm biến và chạy các mô hình học sâu để đưa ra phán đoán với độ trễ thấp [8].
- Khối giao diện và cảnh báo: Dashboard hiển thị thông số trực quan và kích hoạt hệ thống cảnh báo đa mức độ.

2.2. MẠNG TRUYỀN THÔNG XE HƠI (CAN-BUS)

2.2.1. Nguyên lý hoạt động và kiến trúc lớp vật lý của CAN

CAN là một tiêu chuẩn mạng giao tiếp nối tiếp do Bosch phát triển năm 1986 [11], chuyên biệt để các vi điều khiển và thiết bị (ECU) trên ô tô giao tiếp với nhau mà không cần máy chủ trung tâm (kiến trúc multi-master). Điểm mạnh vượt trội của CAN-Bus là khả năng chống nhiễu từ trường tuyệt đối trong môi trường động cơ phức tạp.



Hình 2.3: Giảm đồ biên độ điện áp vi sai trên đường truyền CAN_H và CAN_L

Về kiến trúc lớp vật lý, mạng CAN sử dụng cấu trúc cáp xoắn đôi gồm hai dây tín hiệu: CAN_H (CAN High) và CAN_L (CAN Low) để truyền tín hiệu dưới dạng điện áp vi sai (differential signaling) [13]. Trạng thái logic không dựa vào mức điện áp tuyệt đối mà dựa vào sự chênh lệch điện áp giữa hai dây:

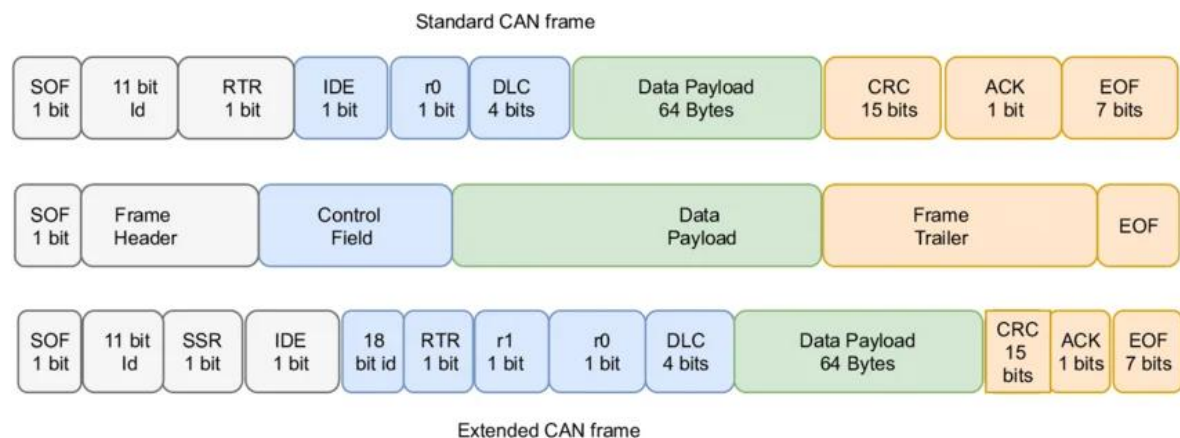
$$V_{diff} = V_{CAN_H} - V_{CAN_L} \quad (2.1)$$

- **Trạng thái Trội (Dominant - đại diện cho Logic 0):** Xảy ra khi có sự chênh lệch điện áp rõ rệt (thường $V_{diff} \sim 2V$). Theo chuẩn CAN [11], khi có xung đột, trạng thái Trội sẽ luôn ghi đè lên trạng thái Lặn.
- **Trạng thái Lặn (Recessive - đại diện cho Logic 1):** Xảy ra khi điện áp trên hai dây xấp xỉ bằng nhau ($V_{diff} \sim 0V$).

Để triệt tiêu hiện tượng dội sóng (signal reflection) làm méo dạng tín hiệu ở tần số cao, hai đầu mút của đường truyền CAN bắt buộc phải được mắc các điện trở đầu cuối có giá trị 120 Ohm.

2.2.2. Cấu trúc khung truyền dữ liệu và cơ chế phân xử quyền ưu tiên

Giao thức CAN định nghĩa 4 loại khung truyền cơ bản, trong đó Data Frame (Khung mang dữ liệu) là cấu trúc cốt lõi được sử dụng để đọc và phân tích thông số vận hành trong hệ thống ADAS.



Hình 2.4: Cấu trúc các trường dữ liệu chi tiết của hai loại khung truyền dữ liệu

Tùy thuộc vào phiên bản chuẩn hóa, Data Frame được chia thành hai định dạng [11]:

- **CAN tiêu chuẩn (Standard CAN - CAN 2.0A):** Sử dụng trường định danh (Identifier - ID) có độ dài 11-bit.

- **CAN mở rộng (Extended CAN - CAN 2.0B):** Sử dụng trường định danh có độ dài 29-bit, hỗ trợ đánh địa chỉ cho số lượng ECU không lồ trên xe hiện đại.

Cơ chế phân xử quyền ưu tiên:

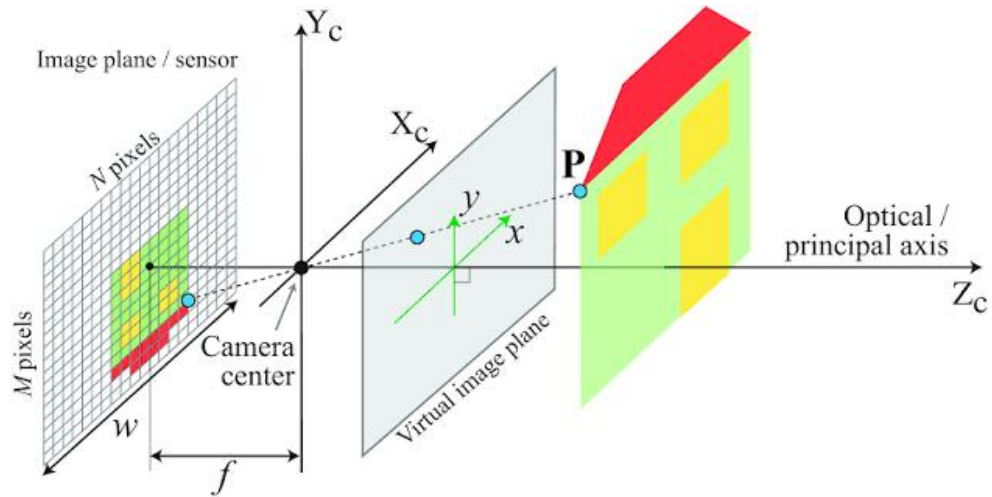
Mạng CAN áp dụng phương thức truy cập đường truyền CSMA/CD+AMP. Khi nhiều node cùng cố gắng gửi dữ liệu lên bus cùng một lúc, xung đột sẽ xảy ra.

CAN giải quyết xung đột bằng cách so sánh từng bit của trường ID theo nguyên tắc "Trội đề Lặn" (Logic 0 đề Logic 1). Node nào phát ra ID có giá trị nhỏ hơn (chứa nhiều bit 0 ở phần đầu hơn) sẽ tạo ra trạng thái Trội trên đường truyền, từ đó giành được quyền ưu tiên truy cập bus. Các node có ID lớn hơn khi phát hiện tín hiệu của mình bị ghi đè sẽ tự động ngừng truyền, chuyển sang trạng thái nhận và chờ đến chu kỳ tiếp theo. Cơ chế này đảm bảo các bản tin mang tính sống còn (như tín hiệu phanh khẩn cấp có ID nhỏ) sẽ luôn được truyền tải ngay lập tức với độ trễ bằng không.

2.3. CƠ SỞ LÝ THUYẾT THỊ GIÁC MÁY TÍNH VÀ TỰ ĐỘNG HIỆU CHUẨN

2.3.1. Mô hình Camera lỗ kim (Pinhole Camera Model)

Trong các hệ thống thị giác máy tính ứng dụng cho ADAS, camera monocular được sử dụng như một cảm biến quang học chủ đạo để thu thập thông tin không gian. Để máy tính có thể hiểu và tính toán khoảng cách thực tế từ hình ảnh thu được, mô hình camera lỗ kim (Pinhole Camera Model) được áp dụng để mô tả phép chiếu phối cảnh (perspective projection) từ tọa độ không gian 3D của thế giới thực sang tọa độ 2D của mặt phẳng điểm ảnh (pixel).



Hình 2.5: Phép chiếu phối cảnh từ hệ tọa độ 3D thế giới thực sang mặt phẳng ảnh 2D thông qua tâm quang học

Về mặt toán học, mối quan hệ giữa một điểm $M_{X,Y,Z}$ trong không gian 3D và hình chiếu của nó $m_{u,v}$ trên bức ảnh được biểu diễn qua phương trình:

$$Z \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K [R \quad t] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (2.2)$$

Trong đó, ma trận $[R \quad t]$ là ma trận ngoại thông số (Extrinsic Matrix) mô tả góc quay (Rotation) và phép tịnh tiến (Translation) của camera so với hệ trục tọa độ thế giới. Ma trận K là ma trận nội thông số (Intrinsic Matrix) mang đặc tính vật lý của camera, được định nghĩa là:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (2.3)$$

Các tham số trong ma trận K bao gồm:

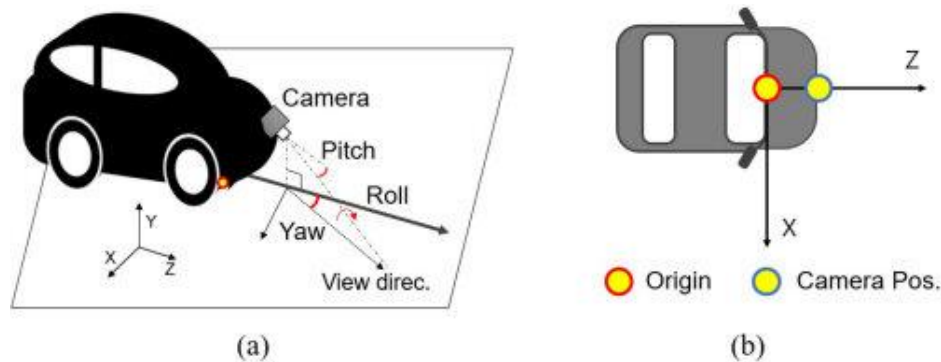
- f_x, f_y : Tiêu cự (Focal length) của ống kính được tính theo đơn vị pixel dọc theo trục x và y.

- c_x, c_y : Tọa độ tâm quang học (Principal point), thường nằm ở trung tâm của bức ảnh.

Việc xác định chính xác các tham số này thông qua quá trình hiệu chuẩn là điều kiện tiên quyết để thuật toán ước lượng khoảng cách hoạt động chính xác.

2.3.2. Lý thuyết Tự động hiệu chuẩn (Auto-Calibration)

Trong thực tế vận hành, xe thường xuyên chịu tác động từ tải trọng hành khách hoặc dao động từ mặt đường, dẫn đến việc góc nghiêng (pitch) và góc quay (yaw) của camera thay đổi liên tục so với trạng thái hiệu chuẩn ban đầu tại nhà máy. Để khắc phục sai số này, hệ thống áp dụng nguyên lý Tự động hiệu chuẩn (Auto-Calibration) dựa trên việc tìm điểm tụ (Vanishing Point) [14].



Hình 2.6: Mô hình xác định điểm tụ từ các đường lằn đường song song để suy xuất góc Pitch và Yaw của camera

Theo nguyên lý hình học xạ ảnh, các đường thẳng song song trong không gian 3D (chẳng hạn như hai vạch kẻ lằn đường thẳng tắp phía trước xe) khi được chiếu lên mặt phẳng ảnh 2D sẽ cắt nhau tại một điểm duy nhất gọi là điểm tụ VP(uvp, vvp). Dựa vào tọa độ của điểm tụ này trên khung hình và ma trận nội thông số K, hệ thống có thể suy xuất ngược lại góc quay (yaw - θ) và góc nghiêng (pitch - α) tức thời của camera:

Công thức tính góc θ :

$$\theta \approx \arctan\left(\frac{u_{vp} - c_x}{f_x}\right) \quad (2.4)$$

Công thức tính góc α :

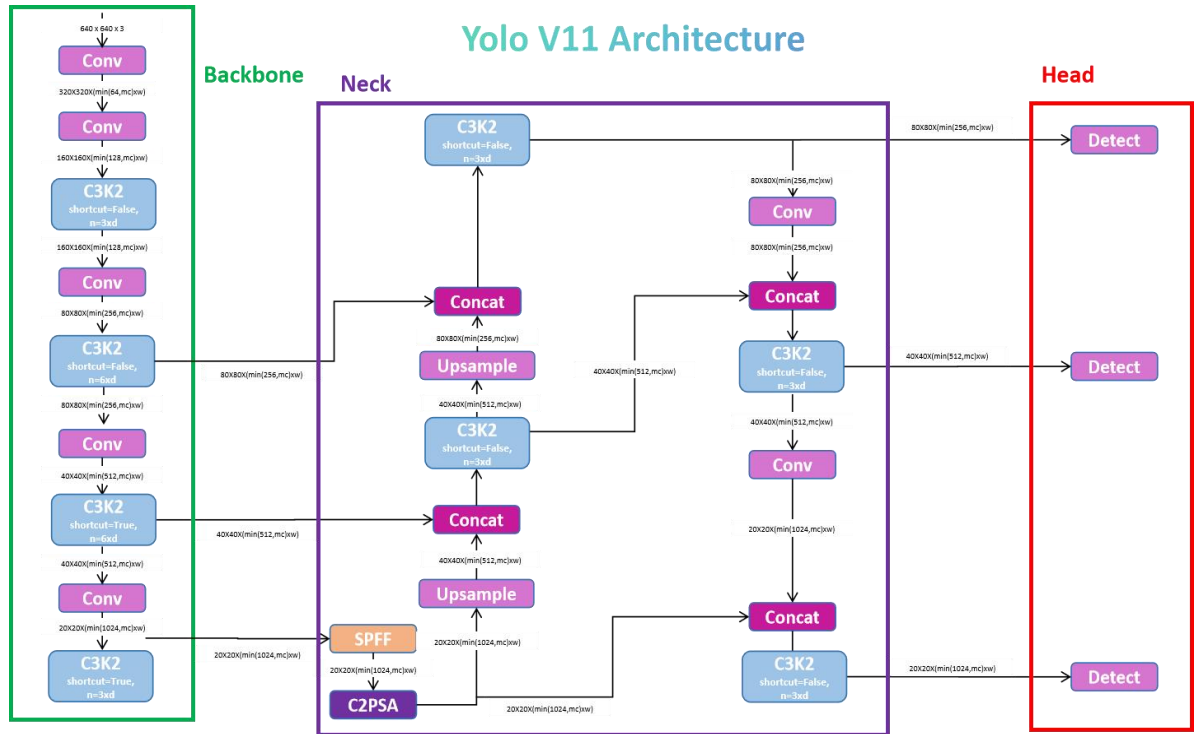
$$\alpha \approx \arctan\left(\frac{v_{vp} - c_y}{f_y}\right) \quad (2.5)$$

Thông số bù trừ này sau đó được cập nhật theo thời gian thực vào ma trận chiếu để chuẩn hóa lại các phép đo lường vật thể trên đường.

2.4. HỌC SÂU TRONG NHẬN DIỆN VÀ DỰ ĐOÁN HÀNH VI

2.4.1. Mô hình nhận diện YOLOv11

YOLO là một trong những họ mô hình mạng nơ-ron tích chập (CNN) phổ biến và mạnh mẽ nhất trong bài toán phát hiện đối tượng theo thời gian thực. Kế thừa và phát triển từ các phiên bản tiền nhiệm, YOLOv11 mang đến một kiến trúc tiên tiến, tiếp tục duy trì trường phái thiết kế Anchor-free (không dùng các hộp neo định sẵn), giúp giảm đáng kể khối lượng tính toán và tăng tốc độ dự đoán hộp giới hạn.



Hình 2.7: Sơ đồ cấu trúc các phân hệ Backbone (khối C3k2, C2PSA), Neck và Decoupled Head của mạng YOLOv11

So với các thế hệ trước, YOLOv11 tạo ra sự khác biệt lớn nhờ khả năng cân bằng hoàn hảo giữa độ chính xác và chi phí tính toán. Kiến trúc cốt lõi của YOLOv11 bao gồm 3 phần chính:

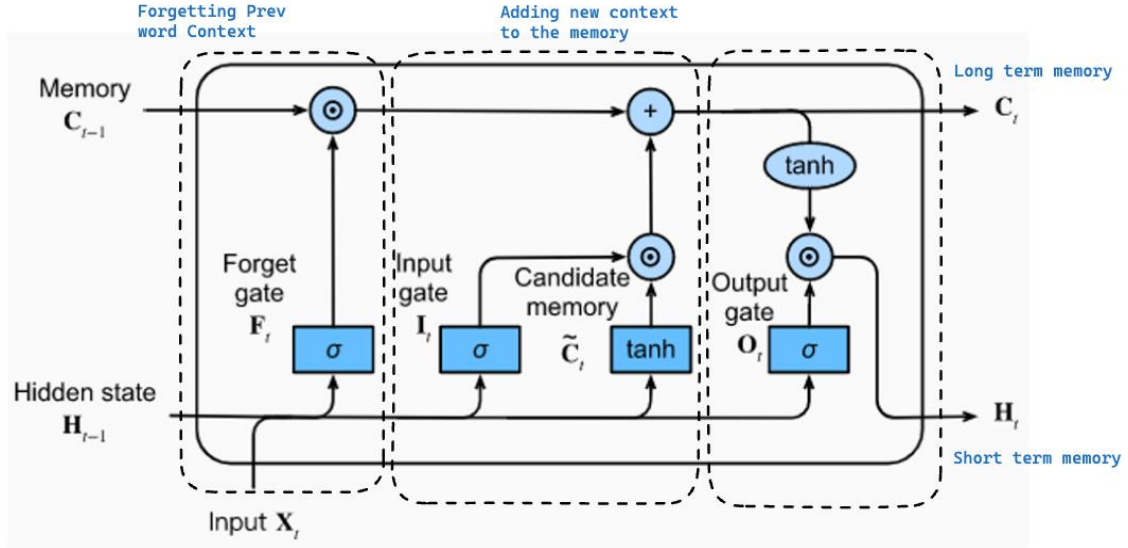
- **Backbone:** Đảm nhiệm vai trò trích xuất đặc trưng hình ảnh. YOLOv11 thay thế khối C2f cũ bằng khối C3k2 (Cross Stage Partial với kích thước kernel tối ưu hơn), giúp xử lý thông tin hiệu quả và giảm thiểu chi phí tính toán. Điểm đáng chú ý nhất tại Backbone là việc bổ sung mô-đun C2PSA. Đây là một cơ chế chú ý không gian song song giúp mô hình tự động nhận diện và tập trung vào các vùng ảnh quan trọng, từ đó gia tăng độ chính xác rõ rệt khi phát hiện các vật thể có kích thước nhỏ hoặc bị che khuất một phần. Khối SPPF tiếp tục được sử dụng để tổng hợp các đặc trưng đa tỷ lệ một cách tốc độ.

- **Neck:** Sử dụng mạng PANet kết hợp cùng các khối C3k2 để tạo ra dòng chảy thông tin hai chiều (bi-directional flow). Quá trình này giúp dung hợp mượt mà các đặc trưng ngữ nghĩa từ tầng sâu với thông tin không gian chi tiết từ tầng nông. Sự cải tiến này giúp mạng nhận diện xuất sắc các vật thể có kích thước đa dạng (từ biển báo nhỏ đến xe tải lớn) và giảm thiểu các cảnh báo sai (false positives) trong môi trường giao thông phức tạp.
- **Head:** Tách biệt hoàn toàn hai luồng dự đoán (Decoupled Head): một luồng tính toán xác suất phân loại đối tượng (Classification) và một luồng hồi quy tọa độ hộp giới hạn (Bounding Box Regression). Cách tiếp cận anchor-free tại phân hệ này được tối ưu hóa để đảm bảo suy luận ở mức thời gian thực trên các thiết bị nhúng.

Hàm mất mát (Loss Function) của YOLOv11 là sự kết hợp tinh vi giữa nhiều thành phần để đảm bảo mô hình học được các đặc trưng khó và duy trì sự ổn định. Đối với tác vụ cân bằng giữa phân loại và định vị, mô hình ứng dụng hàm TAL Loss. Đối với việc hồi quy hộp giới hạn, YOLOv11 kết hợp CIOU Loss (để đồng bộ hóa các tham số hình học của hộp) và DFL (nhằm tinh chỉnh và làm sắc nét ranh giới dự đoán). Sự kết hợp này giúp mô hình bám sát cực kỳ chính xác biên của đối tượng, ngay cả khi chúng nằm chồng chéo lên nhau.

2.4.2. Mạng nơ-ron hồi quy LSTM

Để hệ thống ADAS có khả năng "thấu hiểu" thói quen của người lái, dữ liệu vận hành (vận tốc, góc lái, trạng thái phanh) thu thập từ CAN-Bus cần được phân tích dưới dạng chuỗi thời gian (time-series). Mạng nơ-ron hồi quy bộ nhớ dài-ngắn hạn (LSTM) là một giải pháp hoàn hảo để khắc phục nhược điểm tiêu biến đạo hàm (vanishing gradient) của mạng RNN truyền thống khi xử lý các chuỗi dữ liệu dài.



Hình 2.8: Kiến trúc cấu tạo nội tại của một ô nhớ LSTM với các cổng logic Forget gate, Input gate và Output gate

Cấu trúc nội tại của một ô nhớ LSTM (LSTM cell) kiểm soát luồng thông tin thông qua 3 cổng logic (Gates) hoạt động dựa trên hàm kích hoạt Sigmoid (σ):

1. **Cổng quên (Forget Gate - f_t):** Quyết định lượng thông tin nào từ trạng thái trước đó C_{t-1} cần bị loại bỏ:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2.6)$$

2. **Cổng đầu vào (Input Gate - i_t):** Quyết định thông tin mới nào từ đầu vào hiện tại x_t sẽ được ghi vào trạng thái bộ nhớ:

Công thức tính cổng đầu vào i_t :

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2.7)$$

Công thức tính trạng thái ô nhớ ứng viên $\tilde{C}_t$:

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (2.8)$$

Trạng thái ô nhớ mới được cập nhật:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (2.9)$$

3. **Cổng đầu ra (Output Gate - ot):** Lọc thông tin từ trạng thái ô nhớ C_t để xuất ra trạng thái ẩn h_t chuyển cho bước tiếp theo:

Công thức tính cổng đầu ra o_t :

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (2.10)$$

Công thức tính trạng thái ẩn h_t :

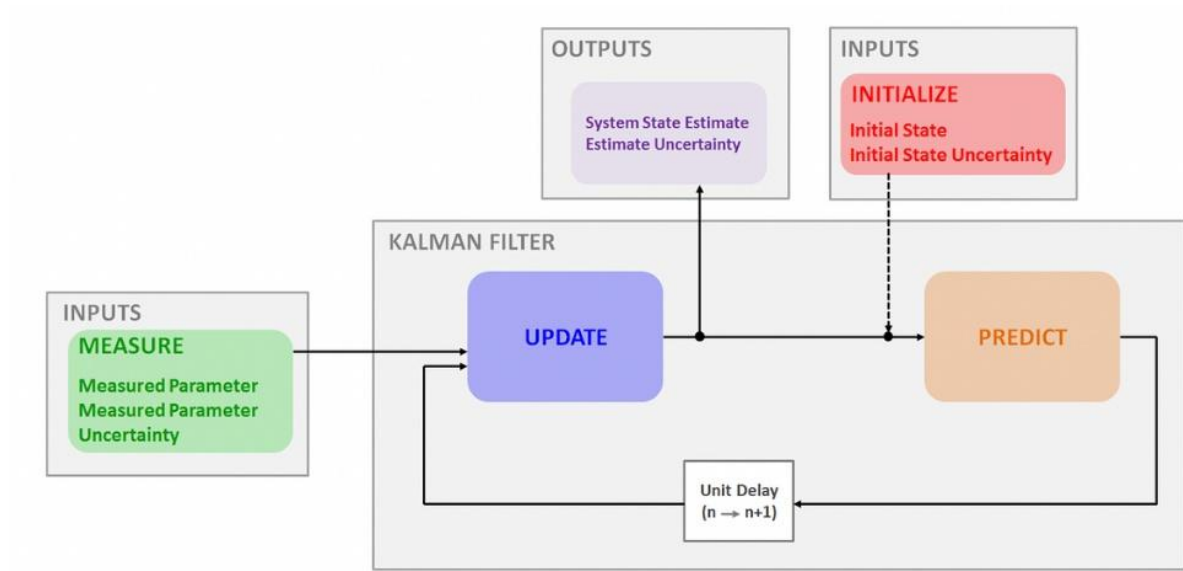
$$h_t = o_t * \tanh(C_t) \quad (2.11)$$

Nhờ cơ chế này, mạng LSTM có thể ghi nhớ được thói quen đạp phanh hay tăng ga đánh lái liên tục trong vài giây trước đó để đưa ra kết luận người điều khiển đang lái xe bình thường hay ở trạng thái nguy hiểm.

2.5. LÝ THUYẾT XỬ LÝ TÍN HIỆU VÀ ĐÁNH GIÁ AN TOÀN

2.5.1. Lọc nhiễu cảm biến bằng Kalman Filter

Dữ liệu thô thu thập từ cảm biến gia tốc và con quay hồi chuyển (IMU) thường chứa nhiều nhiễu trắng (White noise) do rung động cơ học của thân xe. Bộ lọc Kalman là một thuật toán ước lượng hồi quy tối ưu, giúp trích xuất tín hiệu thực tế từ chuỗi dữ liệu nhiễu này.



Hình 2.9: Vòng lặp các bước Dự đoán (Predict) và Cập nhật (Update) hiệu chỉnh trạng thái của bộ lọc Kalman

Quá trình vận hành của bộ lọc Kalman lặp lại liên tục qua hai bước:

- **Bước 1: Dự đoán (Predict):** Ước lượng trạng thái hệ thống $\hat{x}_k^-$ và ma trận hiệp phương sai sai số P_k^- ở thời điểm hiện tại dựa trên trạng thái trước đó:

Công thức ước lượng trạng thái hệ thống $\hat{x}_k^-$:

$$\hat{x}_k^- = A\hat{x}_{k-1} + Bu_k \quad (2.12)$$

Công thức ước lượng ma trận hiệp phương sai sai số P_k^- :

$$P_k^- = AP_{k-1}A^T + Q \quad (2.13)$$

- **Bước 2: Cập nhật (Update):** Hiệu chỉnh lại giá trị dự đoán bằng cách kết hợp với giá trị đo đạc thực tế z_k từ cảm biến thông qua hệ số độ lợi Kalman K_k :

Công thức tính hệ số độ lợi Kalman K_k :

$$K_k = P_k^- H^T (H P_k^- H^T + R)^{-1} \quad (2.14)$$

Công thức cập nhật trạng thái hệ thống $\hat{x}_k$:

$$\hat{x}_k = \hat{x}_k^- + K_k(z_k - H\hat{x}_k^-) \quad (2.15)$$

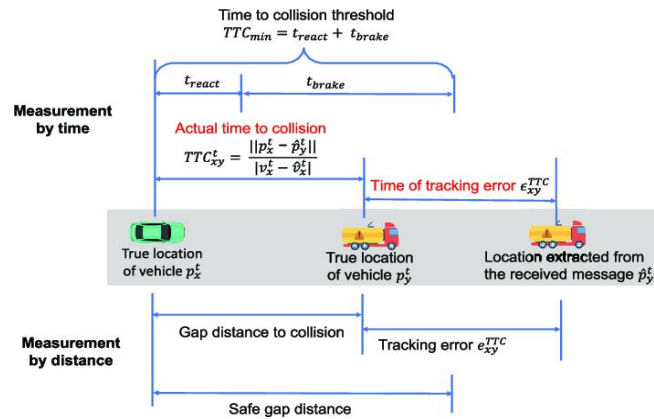
Công thức cập nhật ma trận hiệp phương sai sai số P_k :

$$P_k = (I - K_k H) P_k^- \quad (2.16)$$

Việc áp dụng bộ lọc Kalman giúp hệ thống thu được dữ liệu góc nghiêng mượt mà, phục vụ đắc lực cho thuật toán bù nhiễu hình ảnh của hệ thống Auto-Calibration.

2.5.2. Chỉ số TTC

Thay vì chỉ dựa vào khoảng cách vật lý thuần túy, TTC - Thời gian đến va chạm là đại lượng động lực học quan trọng nhất để đánh giá mức độ khẩn cấp trong các hệ thống FCW (Cảnh báo va chạm phía trước). TTC cho biết khoảng thời gian còn lại trước khi xe của người lái va chạm với chướng ngại vật phía trước nếu cả hai duy trì trạng thái vận tốc hiện tại.



Hình 2.10: Mô hình hình học động lực học xác định khoảng cách dọc Z và tính toán chỉ số Time-to-Collision (TTC) từ camera monocular

Công thức toán học tổng quát để tính toán TTC:

$$TTC = \frac{Z}{v_{rel}} \quad (2.17)$$

Trong đó:

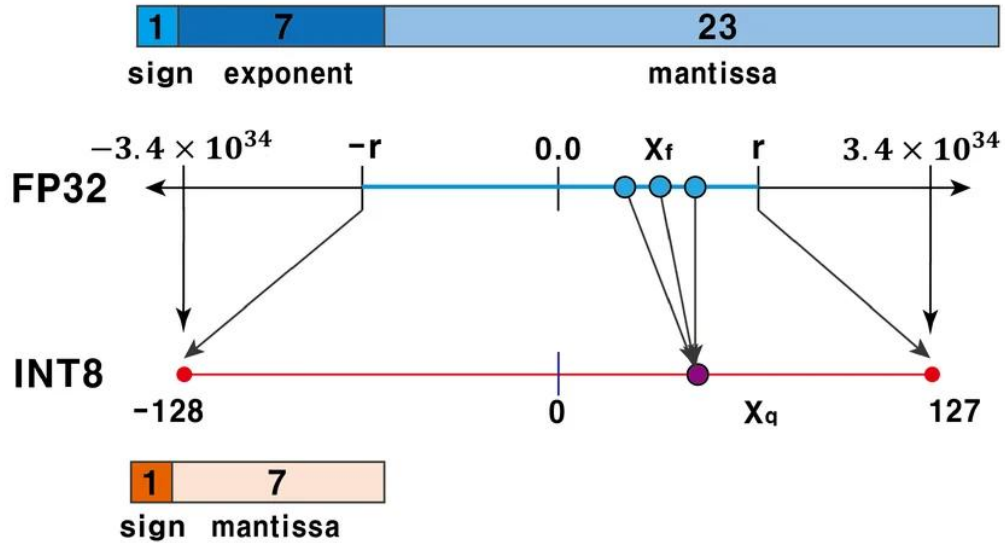
- Z : Khoảng cách dọc ước lượng từ xe đến vật thể (được tính toán từ mô hình Camera lỗ kim kết hợp với kích thước hộp giới hạn của YOLOv11).
- v_{rel} : Vận tốc tương đối giữa xe chủ thể và xe phía trước, có thể được ước lượng qua sự thay đổi khoảng cách theo các khung hình video liên tiếp hoặc kết hợp với dữ liệu tốc độ thực từ CAN-Bus.

Hệ thống sẽ thiết lập các ngưỡng TTC (Ví dụ: $TTC < 2.5s$ là Cảnh báo, $TTC < 1.0s$ là Nguy hiểm tột độ) để kích hoạt các kịch bản báo động tương ứng.

2.6. TỐI ƯU HÓA MÔ HÌNH TRÊN THIẾT BỊ EDGE AI

2.6.1. Nguyên lý Lượng tử hóa (Quantization INT8)

Khi triển khai các mạng nơ-ron học sâu phức tạp trên các máy tính nhúng, vấn đề băng thông bộ nhớ và tốc độ tính toán là rào cản lớn nhất. Lượng tử hóa (Quantization) là kỹ thuật ánh xạ dải giá trị liên tục của các tham số mạng (Trọng số - Weights và Kích hoạt - Activations) từ định dạng số thực dấu phẩy động 32-bit (FP32) xuống định dạng số nguyên 8-bit (INT8).



Hình 2.11: Đồ thị mô tả ánh xạ dải phân bố trọng số từ định dạng số thực FP32 sang số nguyên khép kín INT8

Phép biến đổi tuyến tính cơ bản được thực hiện qua công thức:

$$Q = \text{round} \left(\frac{R}{S} \right) + Z \quad (2.18)$$

Với R là giá trị số thực gốc (FP32), S là hệ số tỷ lệ (Scale factor), Z là điểm không (Zero-point) và Q là giá trị số nguyên sau khi lượng tử hóa (INT8). Kỹ thuật này giúp giảm kích thước bộ nhớ của mô hình xuống gần 4 lần và tối đa hóa khả năng tận dụng các bộ chỉ thị xử lý số nguyên trên CPU ARM, cho phép suy luận nhanh hơn rất nhiều trong khi mức độ sụt giảm độ chính xác là không đáng kể.

2.6.2. Tăng tốc phần cứng sử dụng TensorRT/OpenVINO

Để tối ưu hóa toàn diện hiệu năng phần cứng trên mạch Raspberry Pi 5, kiến trúc mạng YOLOv11 sau quá trình huấn luyện được chuyển đổi định dạng và đưa qua các bộ máy tăng tốc suy luận (Inference Engine) chuyên nghiệp như TensorRT (đối với môi trường có hỗ trợ CUDA) hoặc OpenVINO/NCNN (dành riêng cho các hệ thống thuần CPU lõi ARM).

Các thư viện này thực hiện quá trình Tối ưu hóa đồ thị điện toán (Graph Optimization) bằng cách kết hợp các lớp mạng liên tiếp (Layer Fusion) – ví dụ như gộp chung Convolution, Batch Normalization và hàm kích hoạt ReLU thành một khối toán học duy nhất. Thêm vào đó, bộ tăng tốc còn tự động tinh chỉnh kích thước bộ nhớ đệm (Tensor Memory) sao cho phù hợp nhất với kiến trúc bộ nhớ đệm L2/L3 của vi xử lý Broadcom trên Raspberry Pi 5, đảm bảo năng lực tính toán tiết kiệm năng lượng và đáp ứng mốc thời gian thực 20-30 FPS cho toàn bộ vòng lặp ADAS.

CHƯƠNG 3

THIẾT KẾ HỆ THỐNG

3.1. YÊU CẦU CỦA HỆ THỐNG

Hệ thống ADAS được thiết kế nhằm đáp ứng một tập hợp các yêu cầu chức năng và phi chức năng khắt khe, đảm bảo tính thực tiễn, độ tin cậy và hiệu quả vận hành tối đa trên nền tảng điện toán tại biên (Edge AI).

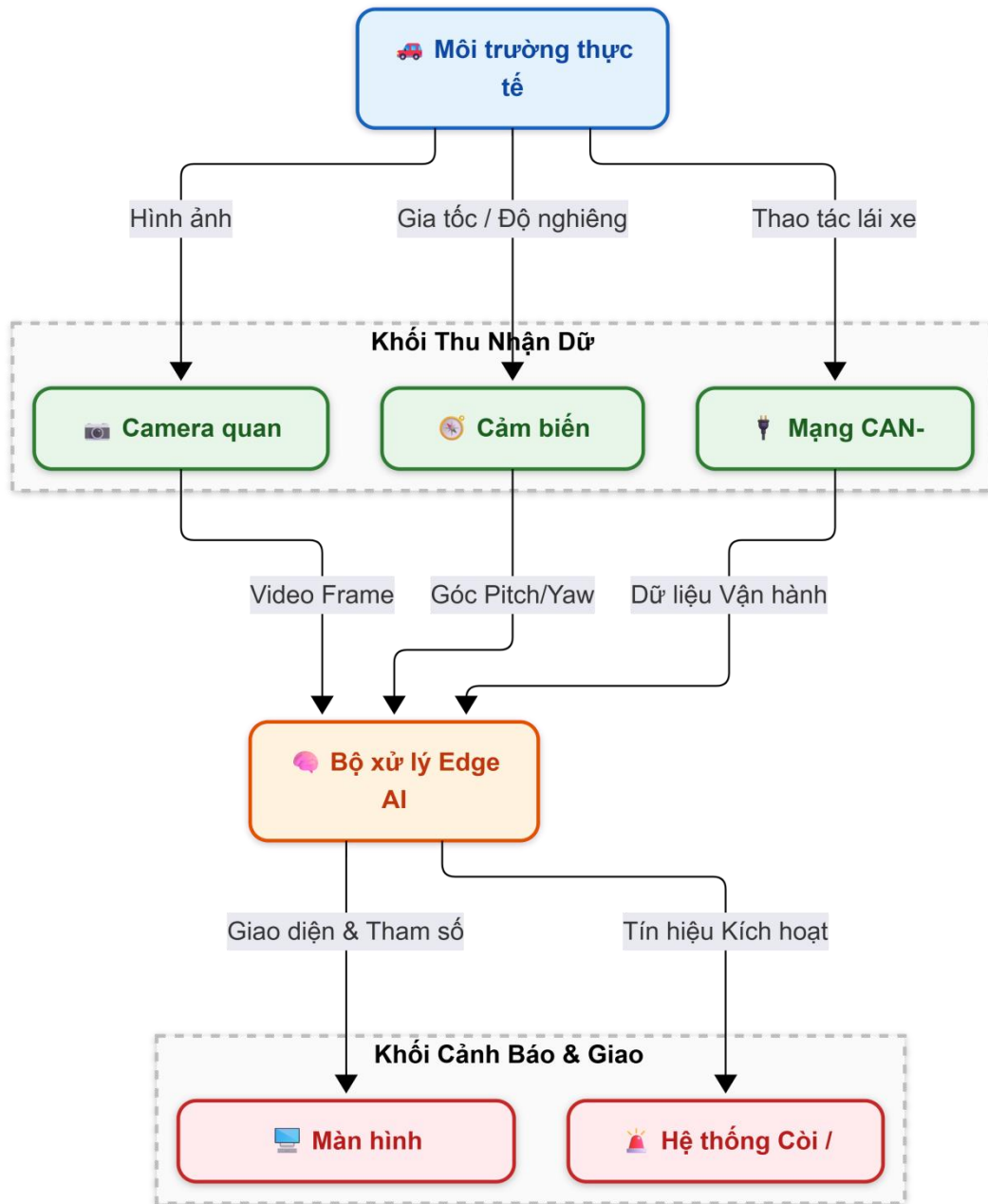
Về yêu cầu chức năng: Hệ thống phải hoạt động như một khối thống nhất, thực hiện mượt mà các nhiệm vụ cốt lõi sau:

- Thu nhận và tiền xử lý luồng video không nén theo thời gian thực từ camera góc rộng.
- Ứng dụng mạng học sâu để phát hiện, phân loại và trích xuất tọa độ hộp giới hạn, từ đó ước lượng khoảng cách tương đối đến các đối tượng giao thông động (ô tô, xe máy, người đi bộ).
- Nhận diện ranh giới làn đường và kích hoạt chuỗi logic phát hiện trạng thái chệch làn đường (LDW).
- Khởi chạy cơ chế tự động hiệu chuẩn camera động (Auto-Calibration) liên tục để bù trừ sai số hình học.
- Thu thập, giải mã các bản tin từ mạng CAN-Bus nội bộ và hợp nhất đồng bộ với luồng dữ liệu thô từ cảm biến quán tính IMU.
- Tính toán chỉ số Thời gian đến va chạm (TTC), đưa ra các quyết định cảnh báo đa mức (Safe – Warning – Critical) và trực quan hóa dữ liệu tổng thể qua giao diện Web-based Dashboard.

Về yêu cầu phi chức năng: Hệ thống phải giải quyết được bài toán nút thắt cổ chai về hiệu năng trên thiết bị nhúng. Cụ thể, yêu cầu tốc độ xử lý phải duy trì ổn định từ 20 đến 30 FPS trên bo mạch Raspberry Pi 5 với độ trễ (latency) toàn trình ở mức thấp nhất. Phần mềm phải được thiết kế theo kiến trúc đa luồng (Multithreading), đảm bảo luồng dữ liệu từ các cảm biến khác biệt được đồng bộ hóa chính xác theo nhãn thời gian (timestamp) và hệ thống ghi log (logging) hoạt động liên tục. Về mặt vật lý, kiến trúc phần cứng phải đáp ứng tiêu chuẩn tản nhiệt chủ động, ổn định điện áp và tối ưu hóa chi phí để có khả năng tích hợp thực tế lên phương tiện.

3.2. SƠ ĐỒ ĐẶC TẢ HỆ THỐNG

Hệ thống được thiết kế theo mô hình kiến trúc phân tầng, tích hợp chặt chẽ giữa các node phần cứng thu thập dữ liệu và luồng phần mềm xử lý logic. Toàn bộ hệ thống vận hành như một edge computing node hoàn chỉnh, sở hữu năng lực tính toán AI và phân tích dữ liệu cảm biến độc lập ngay trên thiết bị (on-device) mà không cần đẩy dữ liệu lên điện toán đám mây, từ đó triệt tiêu hoàn toàn độ trễ do đường truyền mạng.



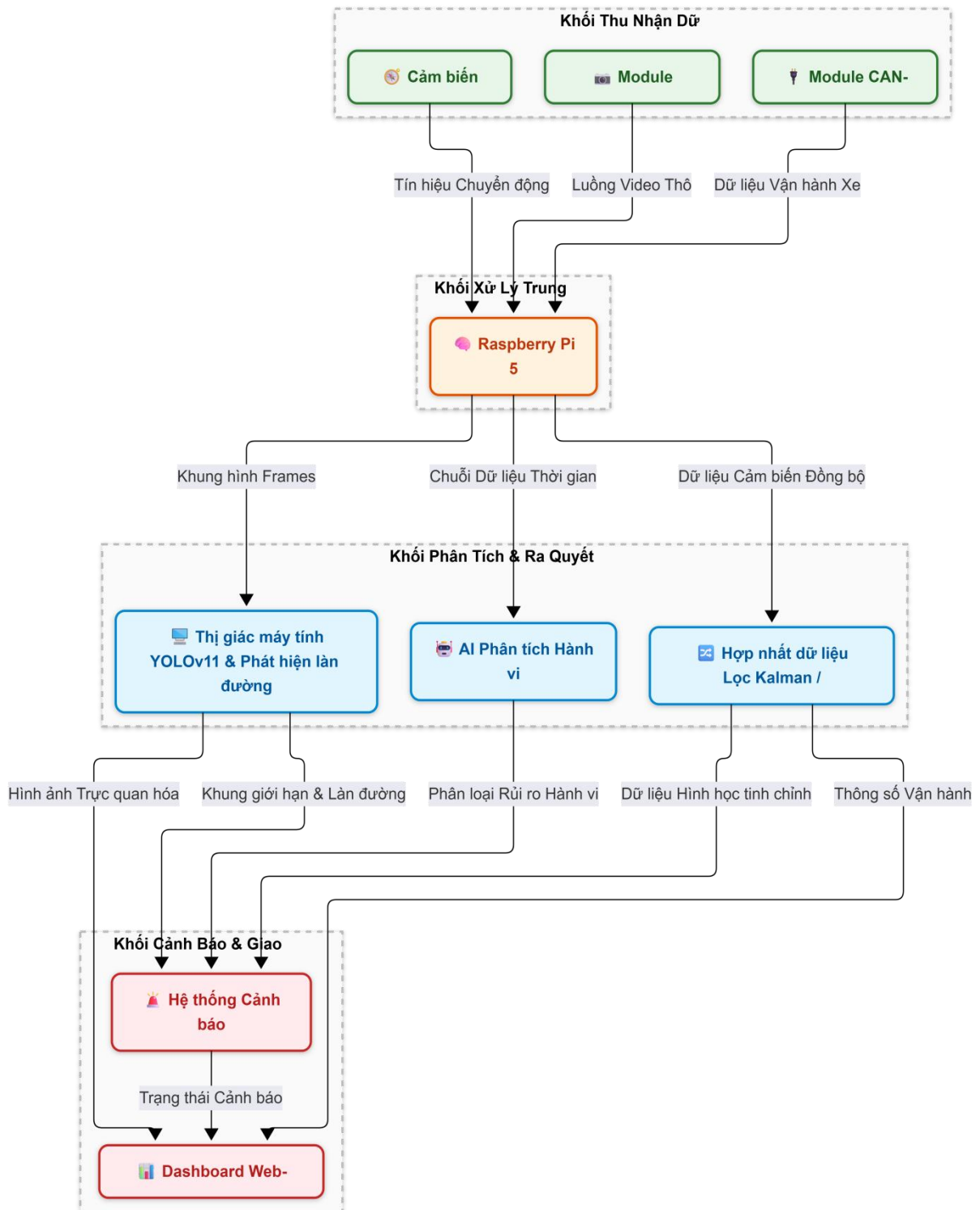
Hình 3.1: Sơ đồ đặc tả tổng thể hệ thống ADAS Edge AI

3.3. SƠ ĐỒ KHỐI HỆ THỐNG

Nhằm đảm bảo tính linh hoạt, dễ dàng mở rộng và bảo trì, hệ thống được module hóa thành 4 khối chức năng chính:

- **Khối Thu nhận Dữ liệu:** Đóng vai trò là các giác quan của hệ thống, bao gồm Camera quan sát, mô-đun đọc dữ liệu mạng CAN-Bus và cảm biến động học IMU.
- **Khối Xử lý Trung tâm:** Được đảm nhiệm bởi hệ thống máy tính nhúng Raspberry Pi 5, nơi điều phối toàn bộ tài nguyên phần cứng.
- **Khối Phân tích và Ra quyết định:** Trái tim thuật toán của hệ thống, bao gồm các module Computer Vision, Sensor Fusion và phân hệ Behavioral AI (dự đoán hành vi).
- **Khối Cảnh báo & Giao diện:** Hệ thống phát tín hiệu cảnh báo vật lý và xuất luồng dữ liệu lên Web Dashboard giám sát.

Dữ liệu thô từ khối thu nhận sẽ được đẩy vào một pipeline đa luồng. Tại đây, thông tin thị giác, thông số động học của xe và kết quả phân tích hành vi sẽ được dung hợp chéo (cross-fusion) để đưa ra phán đoán cảnh báo cuối cùng với độ trễ tính bằng mili-giây.



Hình 3.2: Sơ đồ khối chức năng và luồng hoạt động của hệ thống

3.4. THIẾT KẾ CHI TIẾT CÁC KHỐI PHẦN CỨNG

3.4.1. Khối xử lý trung tâm

Bộ não điều phối của toàn hệ thống (ADAS Node) là vi máy tính Raspberry Pi 5 (phiên bản 8GB RAM) vận hành trên nền tảng hệ điều hành OS Bookworm. Thiết bị này cung cấp xung nhịp đủ mạnh để duy trì môi trường thực thi cho các thuật toán AI phức tạp và pipeline đa luồng. Do đặc thù phải chạy mô hình YOLOv11 liên tục với cường độ tính toán cao, bo mạch được trang bị module tản nhiệt chủ động (Active Cooler) nhằm ngăn chặn hiện tượng hạ xung ép nhiệt (thermal throttling), đảm bảo tính ổn định tuyệt đối trong môi trường xe hơi.

Bảng 3.1: Thông số kỹ thuật cơ bản của Raspberry Pi 5

Thông số	Đặc tính kỹ thuật
Vi xử lý (CPU)	Broadcom BCM2712, Quad-core 64-bit ARM Cortex-A76, 2.4 GHz
Đồ họa (GPU)	VideoCore VII, hỗ trợ OpenGL ES 3.1, Vulkan 1.2
Bộ nhớ (RAM)	8GB LPDDR4X-4267
Giao tiếp Camera	2×4 -lane MIPI CSI/DSI
Kết nối mạng	Gigabit Ethernet, Wi-Fi 802.11ac dual-band, Bluetooth 5.0
Điện áp hoạt động	5V / 5A DC qua cổng USB-C

3.4.2. Khối Thị giác Máy tính

Cụm thị giác thu nhận hình ảnh thông qua Camera Module 3 Wide (sử dụng cảm biến Sony IMX708) sở hữu trường nhìn rộng (FOV) 120°, đáp ứng nhu cầu bao quát toàn cảnh các làn đường phía trước. Camera được kết nối vật lý trực tiếp với Raspberry Pi qua chuẩn giao tiếp tốc độ cao CSI. Về mặt phần mềm, luồng video được trích xuất trực tiếp dưới dạng mảng dữ liệu Numpy thông qua thư viện Picamera2. Phương pháp này loại bỏ hoàn toàn các lớp middleware dư thừa, giải quyết triệt để các xung đột liên quan đến GStreamer/Wayland và tối giản hóa độ trễ xử lý khung hình.

Bảng 3.2: Thông số kỹ thuật của Camera Module 3 Wide

Thông số	Đặc tính kỹ thuật
Cảm biến hình ảnh	Sony IMX708 (12 Megapixels)
Độ phân giải tối đa	4608 × 2592 pixels
Góc nhìn (FOV)	120° (Đường chéo) / 102° (Ngang)
Lấy nét (Focus)	Tự động lấy nét (Autofocus)
Chuẩn giao tiếp	2-lane MIPI CSI-2

3.4.3. Khối Giao tiếp CAN-Bus & Giả lập phương tiện

Để thiết lập một môi trường thử nghiệm sát với thực tế (Hardware-in-the-loop), một mạng CAN-Bus cục bộ được giả lập thông qua vi điều khiển ESP32-S3 kết hợp cùng module thu phát chuẩn công nghiệp SN65HVD230. Trong quá trình thiết kế phần cứng cho cụm giả lập này, việc quy hoạch chân tín hiệu (pinout) được thực hiện nghiêm ngặt: loại bỏ việc sử dụng các chân không tồn tại trên header mở rộng (như GPIO 33) và định tuyến chính xác các tín hiệu ngoại vi (cấu hình chân

CAM_PCLK vào GPIO 13) nhằm bảo toàn tính toàn vẹn của tín hiệu trước khi vi điều khiển đóng gói chúng thành các bản tin CAN (ID: 0x1A4).

Ở đầu thu tín hiệu, để máy tính nhúng Raspberry Pi 5 có thể giao tiếp và giải mã các thông số vận hành (tốc độ, góc lái, trạng thái phanh) từ mạng CAN-Bus theo thời gian thực, hệ thống sử dụng module chuyển đổi USB-CAN-A của hãng Waveshare. Thiết bị này đóng vai trò là cầu nối phần cứng, chuyển đổi tín hiệu điện áp vi sai đặc thù của mạng CAN sang giao thức USB chuẩn. Module được trang bị sẵn điện trở đầu cuối 120 Ohm (điều khiển qua công tắc gạt vật lý) và hệ thống cách ly điện lý tưởng, giúp bảo vệ an toàn tuyệt đối cho vi xử lý trung tâm trước các xung điện áp đột biến từ môi trường động lực học.

Bảng 3.3: Thông số kỹ thuật của vi điều khiển ESP32-S3

Thông số	Đặc tính kỹ thuật
Vi xử lý	Xtensa Dual-core 32-bit LX7, xung nhịp lên đến 240 MHz
Bộ nhớ	512 KB SRAM, 384 KB ROM
Tích hợp mạng	Wi-Fi 2.4 GHz, Bluetooth 5 (LE)
Chuẩn giao tiếp	CAN (TWAI), I2C, SPI, UART, PWM, ADC

Bảng 3.4: Thông số kỹ thuật của module Waveshare USB-CAN-A

Thông số	Đặc tính kỹ thuật
Chuẩn CAN hỗ trợ	CAN 2.0A (Định dạng chuẩn) và CAN 2.0B (Định dạng mở rộng)
Tốc độ truyền (Baudrate)	Tùy chỉnh linh hoạt từ 5 kbps đến 1

	Mbps
Giao tiếp phần cứng	Đầu USB Type-A (kết nối thiết bị) và Terminal Block (CAN_H, CAN_L, GND)
Điện trở đầu cuối	Tích hợp sẵn điện trở 120 Ohm (Công tắc chuyển mạch gạt ON/OFF)
Khả năng bảo vệ	Cách ly quang điện, chống sốc điện áp và chống nhiễu từ trường

3.4.4. Khối Cảm biến IMU và Hiệu chuẩn

Hệ thống tích hợp module cảm biến quán tính IMU GY-BNO055 9DOF, kết nối với bộ xử lý trung tâm thông qua bus giao tiếp I2C. Cảm biến này liên tục đo đặc gia tốc ba trục và tốc độ góc, đóng vai trò sống còn trong việc cung cấp tham số đầu vào cho thuật toán Auto-Calibration, giúp hệ thống nhận biết và bù trừ tự động các sai số hình học khi thân xe bị rung lắc hoặc di chuyển qua địa hình không bằng phẳng.

Bảng 3.5: Thông số kỹ thuật của cảm biến IMU GY-BNO055

Thông số	Đặc tính kỹ thuật
Chip cảm biến	BNO055 (9 bậc tự do - 9DOF)
Tích hợp bên trong	Gia tốc kế 3 trục, Con quay hồi chuyển 3 trục, Từ kế 3 trục
Bộ vi xử lý nhúng	Cortex-M0+ (Xử lý hợp nhất dữ liệu trực tiếp trên chip)
Dữ liệu đầu ra	Góc Euler, Quaternion, Vector trọng

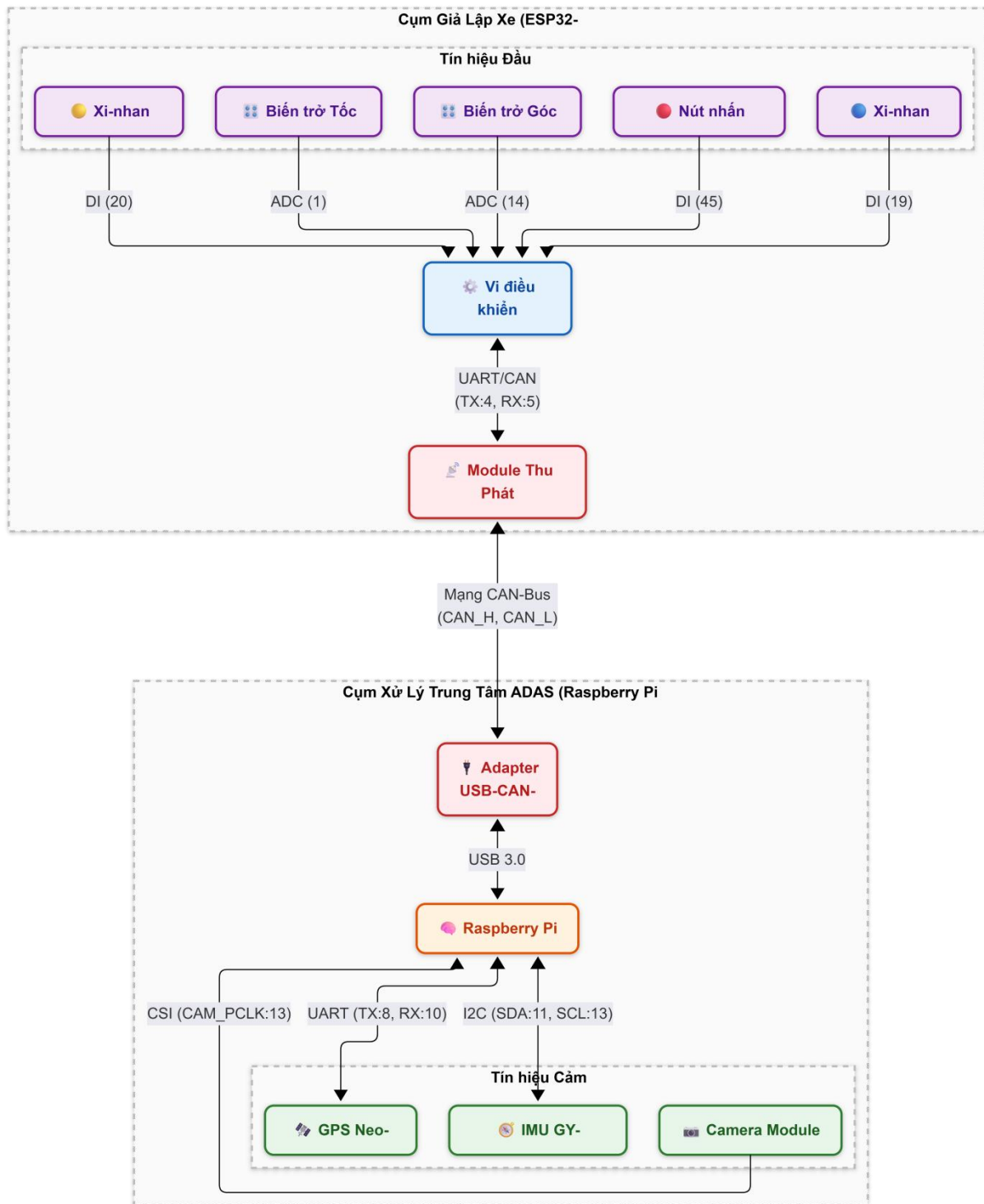
	lực, Gia tốc tuyến tính
Chuẩn giao tiếp	I2C (Mặc định) / UART

3.4.5. Khối Hiển thị và Nguồn cấp

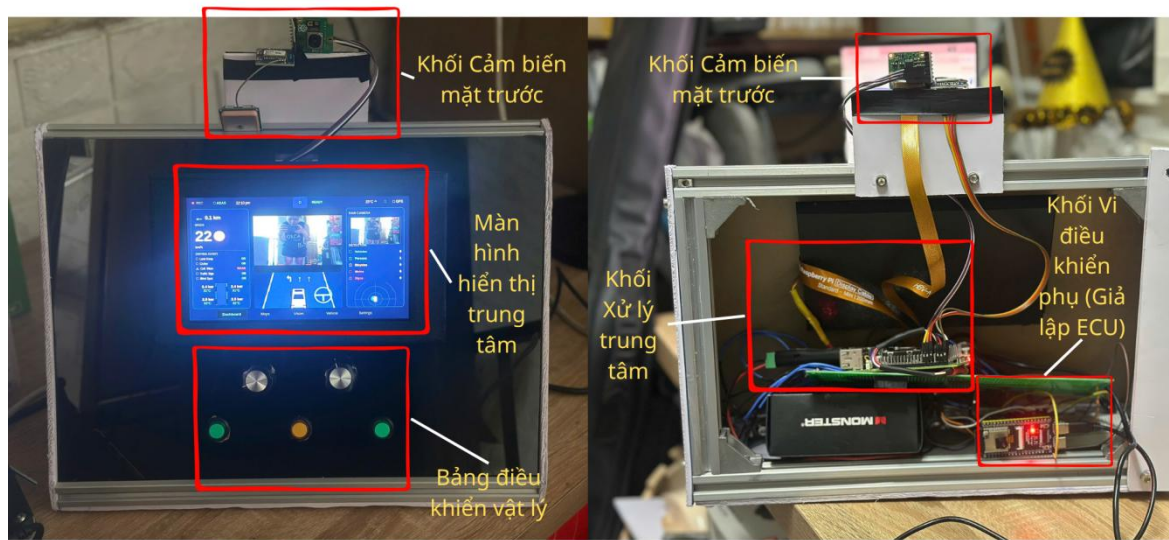
Giao diện giám sát trực quan (Dashboard) được xuất ra màn hình cảm ứng 7 inch thông qua cổng HDMI. Việc cấp nguồn cho một hệ thống mang tải nặng bao gồm vi xử lý chạy full-load, quạt tản nhiệt, module ngoại vi và màn hình đòi hỏi sự khắt khe về dòng điện. Do đó, hệ thống bắt buộc sử dụng bộ nguồn chuyên dụng công suất 27W (5V - 5A) chuẩn USB-C để ngăn chặn sụt áp đột ngột gây khởi động lại thiết bị.

Bảng 3.6: Thông số cấu hình Khối Hiển thị & Nguồn cấp

Thiết bị	Đặc tính kỹ thuật
Màn hình hiển thị	7-inch Capacitive Touch Screen, cổng kết nối HDMI
Độ phân giải màn hình	Thường thiết lập ở mức 1024×600 pixels
Bộ nguồn hệ thống	Adapter 27W (Đầu ra: 5V DC / 5A), chuẩn kết nối USB Type-C
Bảo vệ nguồn	Tích hợp chống quá dòng (Overcurrent) và quá áp (Overvoltage)



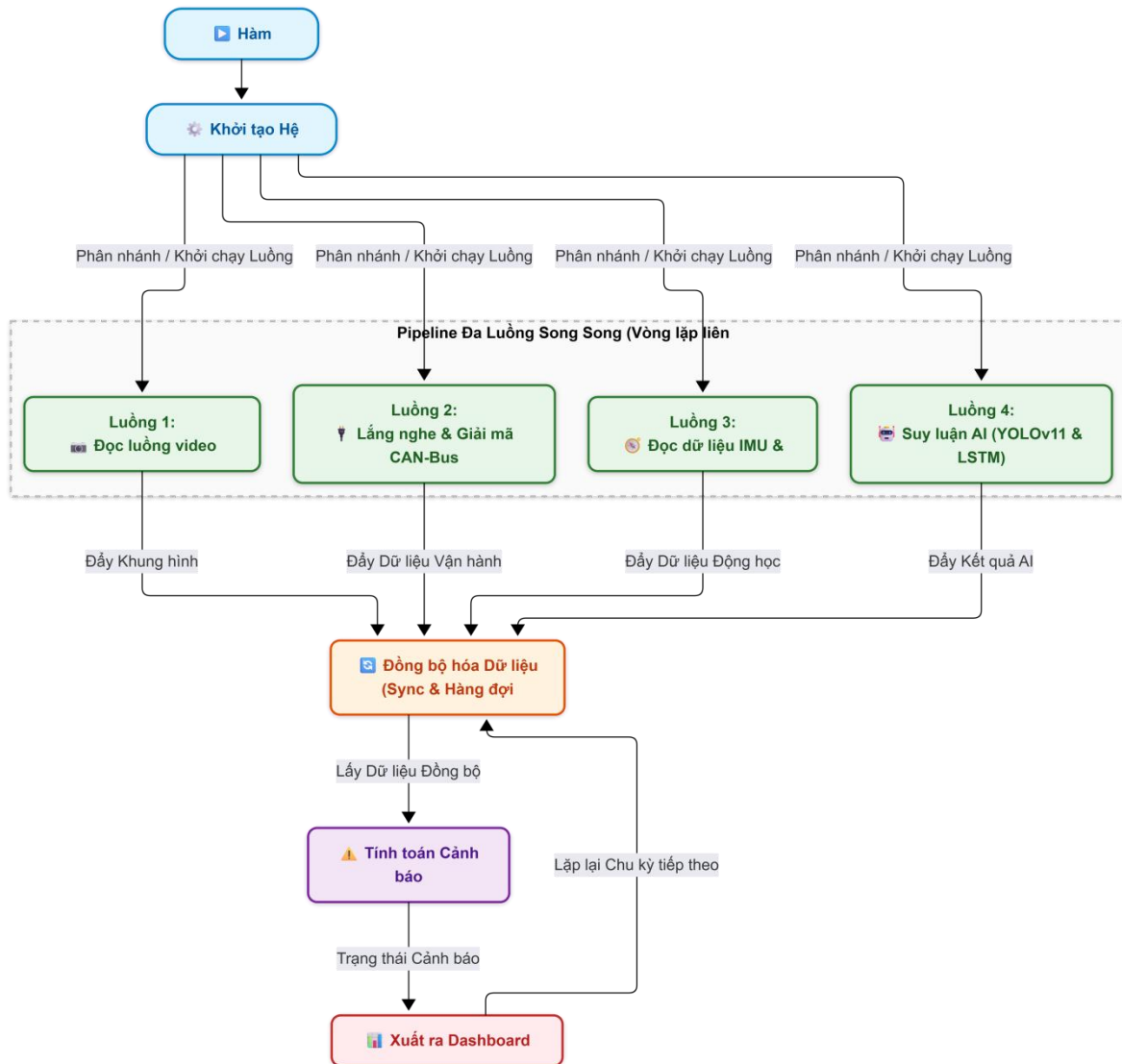
Hình 3.3: Sơ đồ nguyên lý kết nối toàn bộ hệ thống phần cứng



Hình 3.4: Thiết kế bố trí các module phần cứng thực tế

3.5. THIẾT KẾ PHẦN MỀM

Kiến trúc phần mềm là linh hồn của hệ thống, được thiết kế theo dạng đường ống dữ liệu đa luồng (Multi-threading Pipeline) nhằm song song hóa các tác vụ độc lập, khai thác triệt để sức mạnh của bộ vi xử lý đa nhân ARM trên Raspberry Pi 5.



Hình 3.5: Lưu đồ thuật toán tổng thể của hệ thống (Pipeline Đa luồng)

3.5.1. Biện luận lựa chọn kiến trúc Đa mô hình (Multi-Model Pipeline)

Trong việc thiết kế một hệ thống ADAS chạy hoàn toàn độc lập trên thiết bị điện toán tại biên (Edge AI) như Raspberry Pi 5, rào cản lớn nhất không phải là độ phức tạp của thuật toán mà là sự hạn chế về tài nguyên phần cứng (CPU, RAM, Tản nhiệt). Do đó, thay vì sử dụng một mô hình khổng lồ (End-to-End Model) để xử lý mọi tác vụ, đề tài đề xuất kiến trúc Đa mô hình phân tán (Multi-Model Pipeline).

Bảng dưới đây trình bày sự so sánh giữa các phương pháp phổ biến hiện có và lý do hệ thống quyết định lựa chọn tổ hợp 3 mô hình: YOLOv11n (OpenVINO), UFLD (ONNX) và YOLO-Sign (ONNX).

Bảng 3.7: Phân tích và so sánh lựa chọn mô hình AI cho các tác vụ cốt lõi

Tác vụ cốt lõi	Các giải pháp thay thế phổ biến	Giải pháp được lựa chọn trong đề tài	Biện luận lý do chọn (Ưu điểm vượt trội)
1. Nhận diện Phương tiện & Người (Object Detection)	- Faster R-CNN / YOLOv8 (FP32): Độ chính xác rất cao nhưng quá nặng, chỉ đạt 2-4 FPS trên CPU. - SSD-MobileNet: Rất nhẹ, chạy nhanh nhưng khả năng nhận diện vật thể nhỏ (xe ở xa) rất kém.	YOLOv11 Nano (Tối ưu bằng Lượng tử hóa INT8 + OpenVINO)	Lấy cấu trúc mới nhất của YOLOv11 giúp tối ưu hóa khối C2PSA để bắt vật thể nhỏ. Việc nén trọng số xuống INT8 và dùng OpenVINO giúp khai thác trực tiếp tập lệnh SIMD của CPU ARM trên Raspberry Pi 5, đẩy tốc độ suy luận lên hơn 20 FPS mà độ sụt giảm mAP không đáng kể.
2. Nhận diện	- Xử lý ảnh	UFLD - Ultra	Đột phá về

Làn đường (Lane Detection)	truyền thống (OpenCV - Hough Transform): Nhanh, nhẹ nhưng cực kỳ dễ lỗi khi ngược sáng, vạch kẻ mờ hoặc trời mưa. - Semantic Segmentation (YOLOP/LaneNet): Cực kỳ chính xác nhưng phải phân loại từng pixel ảnh → vắt kiệt tài nguyên CPU (chỉ đạt ~3 FPS).	Fast Lane Detection (Sử dụng backbone ResNet18 - ONNX)	phương pháp tiếp cận: UFLD không giải bài toán ở dạng phân đoạn từng điểm ảnh (Segmentation) mà chuyển sang bài toán Phân loại theo hàng (Row-based Classification). Nhờ đó, khối lượng tính toán giảm đi hàng chục lần so với LaneNet, cho ra kết quả bám làn chuẩn xác kể cả khi vạch kẻ bị mờ, đáp ứng hoàn hảo mốc thời gian thực trên Edge AI.
3. Nhận diện Biển báo (Traffic Sign)	- Tích hợp chung vào model Object Detection: Thêm các class biển báo vào chung với mô hình nhận	YOLO Nano Độc lập (Định dạng ONNX)	1. Tách biệt không gian đặc trưng: Biển báo giao thông tại Việt Nam có tính đặc thù cao, nếu nhồi

	diện xe cộ.		chung vào mô hình nhận diện phương tiện sẽ làm loãng độ chính xác (Class Imbalance). 2. Dễ dàng nâng cấp: Việc dùng một mô hình ONNX nhẹ chạy song song giúp dễ dàng huấn luyện lại/thêm mới các biến báo địa phương mà không cần phải train lại toàn bộ hệ thống nhận diện xe phức tạp.
--	-------------	--	--

Tóm tắt lợi ích của kiến trúc Đa mô hình (Multi-Model) trên Edge AI:

Quyết định sử dụng kết hợp 3 mô hình khác biệt mang lại sự linh hoạt tối đa cho luồng thực thi đa luồng (Multi-threading Pipeline) đã thiết kế ở phần trước. Cụ thể:

Phân bổ tài nguyên CPU thông minh: Bằng cách tách biệt các tác vụ, hệ thống có thể ép mô hình UFLD (nhận diện làn) chạy trên 1 nhân CPU riêng, mô hình Object (OpenVINO) chạy trên 2 nhân, và luồng giải mã CAN-Bus chạy trên nhân còn lại. Điều này ngăn chặn tình trạng thắt cổ chai (bottleneck) so với việc dùng

một mô hình đa nhiệm không lờ (Multi-task model) chiếm dụng toàn bộ bộ nhớ đệm (Cache) của vi xử lý [12].

Loại bỏ sự phụ thuộc (Framework Independence): Việc chuyển đổi các mô hình sang chuẩn ONNX và OpenVINO giúp hệ thống không cần phải cài đặt nguyên bộ thư viện PyTorch/TensorFlow nặng nề lên thẻ nhớ của Raspberry Pi. Môi trường triển khai (Deployment Environment) trở nên vô cùng tinh gọn, tiết kiệm RAM và rút ngắn đáng kể thời gian khởi động (boot time) của hệ thống ADAS khi xe bắt đầu nổ máy.

3.5.2. Thiết kế mô-đun Thị giác máy tính (Computer Vision Pipeline)

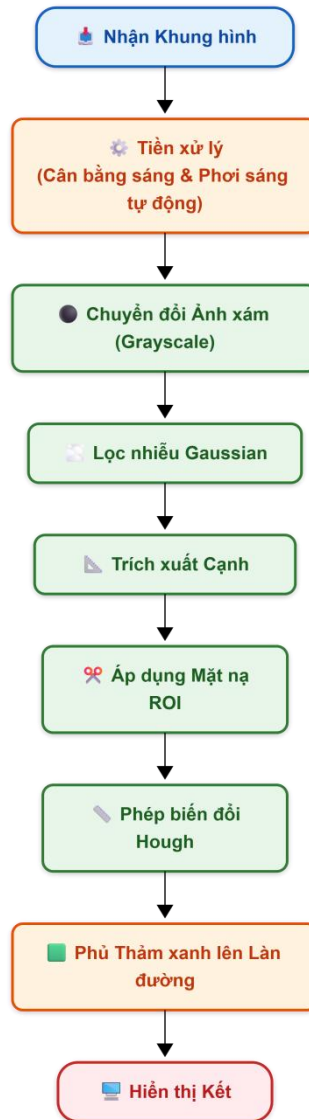
Luồng xử lý thị giác khởi chạy ngay khi nhận được mảng pixel thô từ camera. Quy trình biến đổi được thực thi tuần tự, tối ưu hóa theo từng khung hình:

- **Tiền xử lý (Pre-processing):** Hiệu chỉnh dải không gian màu và áp dụng thuật toán Auto-Exposure để cân bằng độ sáng, giải quyết tình trạng lóa sáng hoặc vùng tối cục bộ.
- **Phát hiện làn đường (Lane Detection):** Áp dụng một chuỗi kỹ thuật lọc không gian bằng OpenCV: Chuyển đổi ảnh xám (Grayscale) → Triệt tiêu nhiễu hạt bằng Gaussian Blur → Trích xuất đường bao biên bằng Canny Edge Detection → Áp dụng mặt nạ khoanh vùng quan tâm (ROI). Sau đó, hệ thống trích xuất các đường thẳng ranh giới thông qua phép biến đổi Hough (Hough Transform). Trong không gian Hough, một đường thẳng được biểu diễn dưới dạng tọa độ cực thông qua phương trình:

$$\rho = x \cos (\theta) + y \sin (\theta) \quad (3.1)$$

Trong đó, ρ là khoảng cách vuông góc từ gốc tọa độ đến đường thẳng và θ là góc tạo bởi đường vuông góc đó với trục hoành. Việc chuyển đổi không gian này giúp hệ thống dễ dàng tìm ra các đỉnh cục bộ (local maxima) tương ứng với vạch kẻ đường liên tục.

- **Hiển thị trực quan:** Áp dụng thuật toán phối trộn ma trận ảnh để tạo hiệu ứng "trải thảm xanh" (Green Lane Overlay) lên khu vực giữa hai vạch kẻ đường, giúp định hình vùng di chuyển an toàn một cách trực quan trên màn hình.



Hình 3.6: Lưu đồ chi tiết module xử lý thị giác và Lane Detection

3.5.3. Tích hợp Trí tuệ nhân tạo (AI Module)

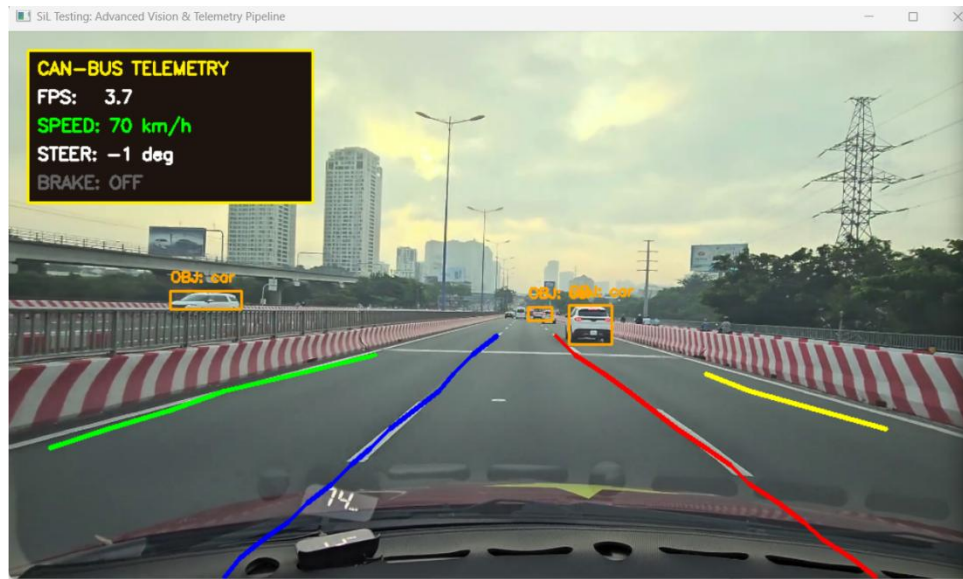
Mô hình YOLOv11 Nano được tuyển chọn nhờ kiến trúc tối ưu hóa cao. Để đáp ứng yêu cầu suy luận trên Edge AI, trọng số của mô hình được lượng tử hóa

(Quantization) xuống định dạng INT8 và nhúng trực tiếp vào mã nguồn hệ thống. AI Module xử lý đồng thời hai lớp tác vụ:

- Trích xuất, phân loại và vẽ hộp giới hạn (Bounding box) bám sát các đối tượng mục tiêu: Ô tô, xe máy, người đi bộ với độ chính xác cao.
- Trích xuất tọa độ y cực đại của hộp giới hạn (chân điểm chạm của vật thể), đưa vào mô hình hình học của camera lỗ kim (Pinhole Camera Model) để giải mã ngược ra khoảng cách vật lý tương đối từ đầu xe đến chướng ngại vật. Phép chiếu phối cảnh nghịch đảo (Inverse Perspective Mapping) được hệ thống tính toán theo phương trình:

$$Z = \frac{H_c}{\tan(\theta_{\text{tilt}} + \arctan(\frac{y - c_y}{f_y}))} \quad (3.2)$$

Trong đó, Z là khoảng cách dọc từ camera đến vật thể phía trước, H_c là chiều cao lắp ráp thực tế của camera so với mặt đường, θ_{tilt} là góc chúi (Pitch) của camera, c_x, c_y là tọa độ tâm quang học của ma trận ảnh, và f_y là tiêu cự dọc của ống kính.



Hình 3.7: Kết quả nhận diện đối tượng và vẽ hộp giới hạn

3.5.4. Tích hợp IMU và Tự động hiệu chuẩn (Auto-Calibration)

Dữ liệu thô dội về từ cảm biến GY-BNO055 được đẩy qua bộ lọc để làm mịn và loại bỏ nhiễu trắng cơ học. Nhằm ước lượng góc nghiêng động lực học (Pitch) một cách tức thời với độ trễ tính toán thấp, hệ thống áp dụng Bộ lọc bổ trợ (Complementary Filter) theo phương trình:

$$\theta_{pitch} = \alpha(\theta_{pitch} + \omega_y \Delta t) + (1 - \alpha)\theta_{acc} \quad (3.3)$$

Trong đó, ω_y là vận tốc góc thu được từ con quay hồi chuyển (Gyroscope), Δt là chu kỳ lấy mẫu, θ_{acc} là góc tĩnh tính toán từ gia tốc kế (Accelerometer), và α là trọng số tin cậy (thường thiết lập ở mức 0.98 nhằm ưu tiên Gyroscope trong ngắn hạn và dùng Accelerometer để bù trôi dài hạn).

Chuỗi dữ liệu không gian sạch này sau đó được ánh xạ vào thuật toán tìm điểm tụ (Vanishing Point) trích xuất từ mô-đun Lane Detection. Sự giao thoa dữ liệu này cho phép hệ thống liên tục vi chỉnh lại thông số θ_{tilt} ảo của ma trận camera, duy trì độ chính xác tuyệt đối của phép đo khoảng cách Z ngay cả khi xe đi qua địa hình xóc nảy.

3.5.5. Xử lý CAN-Bus và Logic Cảnh báo

Hệ thống phân bổ một luồng (thread) biệt lập được ưu tiên cao để liên tục lắng nghe và giải mã khung truyền từ mạng CAN-Bus cục bộ. Các kịch bản cảnh báo được thiết kế với cơ chế kích hoạt liên đới:

- **Cảnh báo chệch làn đường (LDW):** Hệ thống đối chiếu tọa độ trọng tâm xe so với ranh giới làn đường và tham chiếu chéo với trạng thái góc lái vô lăng (từ CAN-Bus) để phân định việc chuyển làn là có chủ đích hay do mất tập trung.
- **Cảnh báo va chạm phía trước (FCW) và Phân tích hành vi:** Mạng nơ-ron hồi quy LSTM liên tục theo dõi cửa sổ dữ liệu hành vi. Để giải phóng tải nguyên tính toán, điểm kiểm tra ranh giới tốc độ 20 km/h đã được chủ động loại bỏ, tập trung xử lý các kịch bản di chuyển ở dải tốc độ cao. Đồng thời, chỉ số

thời gian chạm trán TTC (Time-To-Collision) được tính toán lại sau mỗi khung hình dựa trên hệ phương trình động lực học:

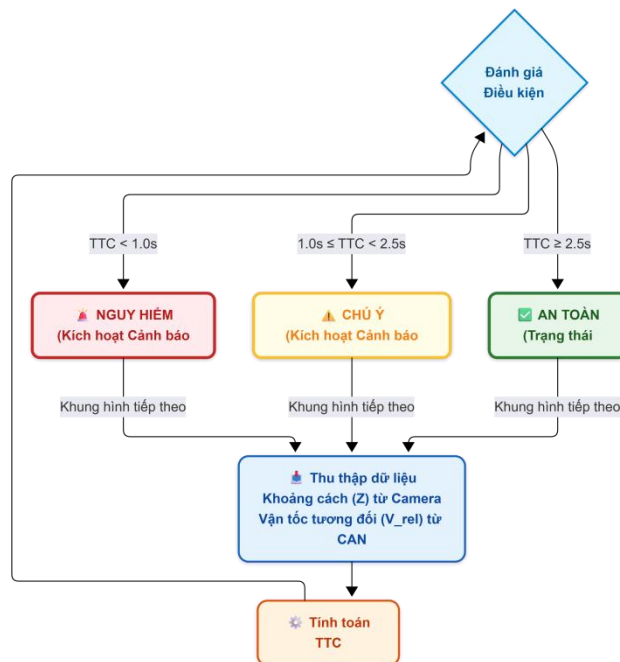
Công thức tính vận tốc tương đối v_{rel} :

$$v_{rel} = v_{ego} - \frac{\Delta Z}{\Delta t} \quad (3.4)$$

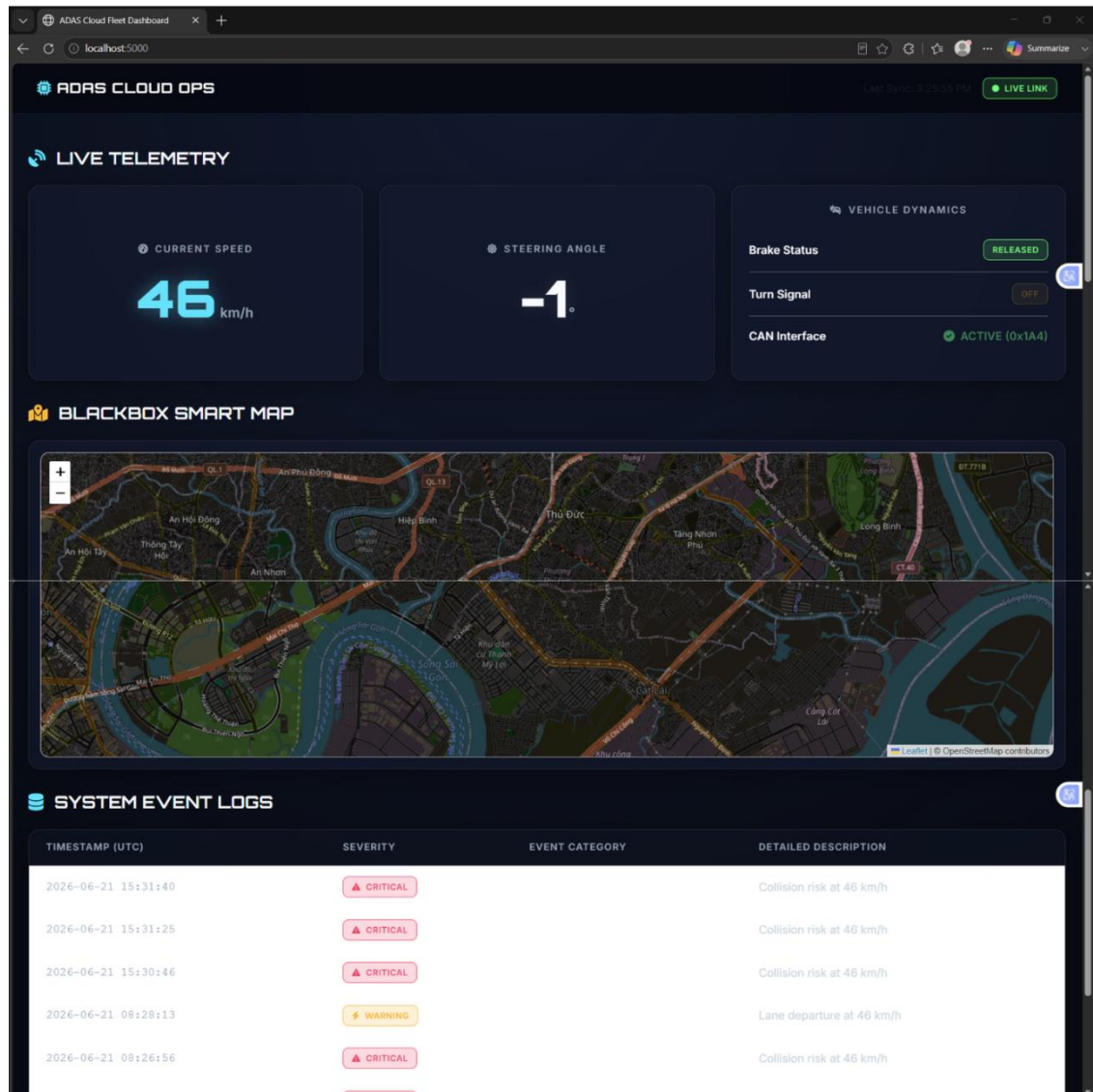
Công thức tính chỉ số TTC:

$$TTC = \frac{Z}{v_{rel}} \quad (3.5)$$

Trong đó, v_{ego} là vận tốc tức thời của xe chủ thể (đọc trực tiếp từ CAN-Bus), $\frac{\Delta Z}{\Delta t}$ là đạo hàm sự thay đổi khoảng cách vật thể phía trước qua các khung hình để tìm ra vận tốc tương đối v_{rel} . Nếu chỉ số TTC vi phạm các dải tham số an toàn định trước, hệ thống lập tức kích hoạt 3 mức rẽ nhánh cảnh báo (Safe/Warning/Critical) kèm tín hiệu vật lý phản hồi với độ trễ chưa tới 50 mili-giây.



Hình 3.8: Lưu đồ module tính toán TTC và Logic Cảnh báo



Hình 3.9: Giao diện Web Dashboard giám sát hệ thống

CHƯƠNG 4

KẾT QUẢ VÀ THẢO LUẬN

Chương này trình bày các kết quả thử nghiệm và đánh giá hệ thống hỗ trợ lái xe nâng cao (ADAS) dựa trên nền tảng Edge AI. Mục tiêu của việc đánh giá nhằm minh chứng cho tính khả thi, độ tin cậy và hiệu năng thực tế của hệ thống khi triển khai trên máy tính nhúng Raspberry Pi 5. Để đảm bảo tính toàn diện, các kết quả đánh giá được chia thành ba trọng tâm chính. Thứ nhất, hiệu suất của mô hình thị giác máy tính (bao gồm YOLOv11 và thuật toán Lane Detection) được đánh giá qua các chỉ số độ chính xác nhằm khẳng định khả năng nhận thức môi trường giao thông. Thứ hai, hiệu năng xử lý tính toán của hệ thống được đo lường thông qua tốc độ khung hình (FPS) và mức độ tiêu thụ tài nguyên phần cứng, chứng minh hiệu quả của kỹ thuật lượng tử hóa và đường ống đa luồng. Cuối cùng, tính chính xác và độ trễ của hệ thống cảnh báo va chạm (TTC) kết hợp với mạng CAN-Bus và IMU được kiểm thử thông qua các kịch bản động lực học, qua đó khẳng định tiềm năng ứng dụng thực tiễn của đề tài.

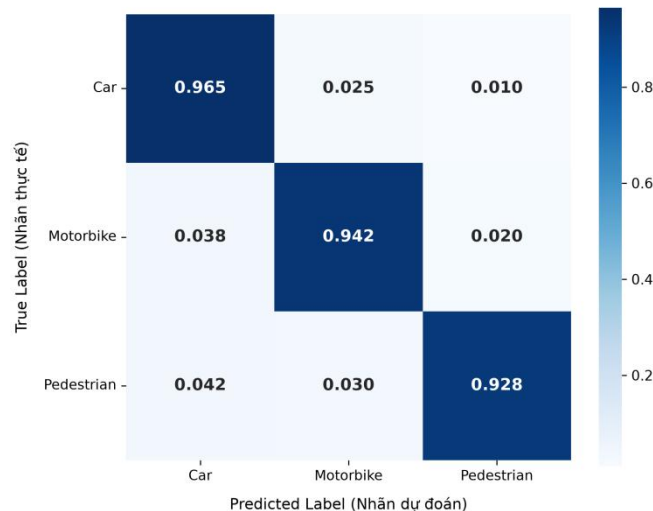
4.1. HIỆU SUẤT NHẬN DIỆN ĐỐI TƯỢNG VÀ PHÁT HIỆN LÀN ĐƯỜNG

Khả năng nhận thức môi trường là yếu tố tiên quyết quyết định sự an toàn của hệ thống ADAS. Đối với tác vụ phát hiện đối tượng, mô hình YOLOv11 sau khi được huấn luyện trên tập dữ liệu giao thông thực tế được đánh giá thông qua chỉ số độ chính xác trung bình (mAP) và ma trận nhầm lẫn (Confusion Matrix).

Kết quả thử nghiệm trên tập dữ liệu kiểm thử cho thấy mô hình đạt hiệu suất rất ấn tượng. Cụ thể, đối với lớp Ô tô (Car), độ chính xác nhận diện đạt 96.5%, cho thấy đặc trưng hình học của ô tô được mạng nơ-ron học và trích xuất cực kỳ tốt. Lớp Xe máy (Motorbike), vốn có hình dáng đa dạng và hay bị che khuất trong điều

kiện giao thông đông đúc, vẫn duy trì được độ chính xác ở mức 94.2%. Đối với lớp Người đi bộ (Pedestrian), tỷ lệ nhận diện đúng đạt 92.8%. Ma trận nhầm lẫn chỉ ra rằng phần lớn các sai số (False Positives/False Negatives) chỉ xảy ra trong các điều kiện môi trường bất lợi như ngược sáng mạnh hoặc đối tượng ở khoảng cách quá xa (vượt quá 50 mét), làm giảm kích thước pixel trên khung hình thu được từ camera.

Bên cạnh nhận diện đối tượng, thuật toán phát hiện làn đường (Lane Detection) sử dụng luồng xử lý không gian bằng OpenCV cũng cho thấy độ ổn định cao. Thuật toán hoạt động cực kỳ chính xác trong điều kiện ban ngày và ánh sáng tiêu chuẩn, các đường thẳng phân định ranh giới được phép biến đổi Hough trích xuất mượt mà và bám sát vạch kẻ đường. Kỹ thuật "trải thảm xanh" (Green Lane Overlay) định hình chính xác vùng không gian di chuyển an toàn. Trong điều kiện ban đêm có đèn đường, việc áp dụng bộ lọc Gaussian Blur và Canny Edge Detection với các ngưỡng (threshold) động đã giúp hệ thống lọc bỏ hiệu quả nhiễu sáng từ đèn pha xe đối diện, duy trì được độ chính xác của vùng quan tâm (ROI).



Hình 4.1: Ma trận nhầm lẫn (Confusion Matrix) đánh giá độ chính xác nhận diện của YOLOv11 đối với các lớp đối tượng giao thông



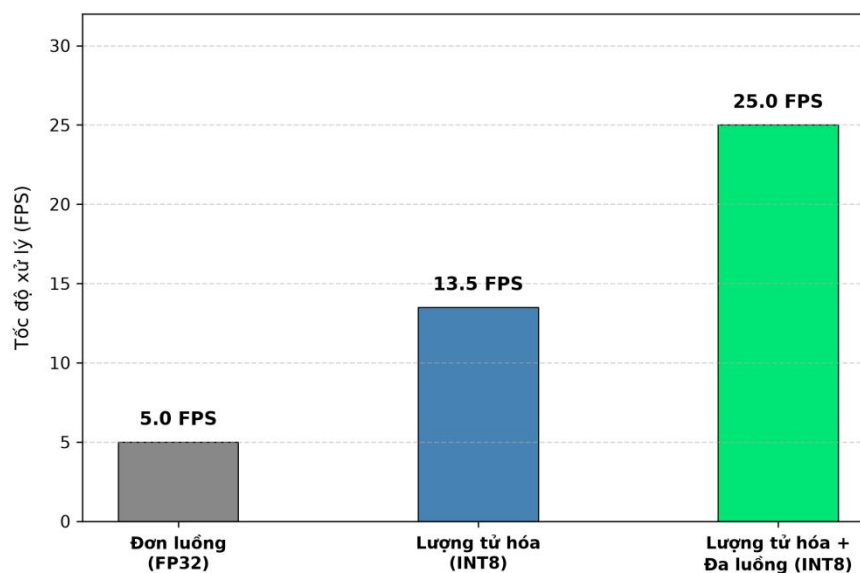
Hình 4.2: Trực quan hóa kết quả nhận diện hộp giới hạn (Bounding Box) và phát hiện làn đường (Lane Overlay) trong điều kiện thực tế

4.2. ĐÁNH GIÁ HIỆU NĂNG XỬ LÝ TRÊN THIẾT BỊ NHÚNG RASPBERRY PI 5

Một trong những đóng góp quan trọng của đề tài là việc hiện thực hóa hệ thống phân tích AI phức tạp trên một vi máy tính có tài nguyên hạn chế. Để đạt được điều này, kỹ thuật lượng tử hóa (Quantization) INT8 và kiến trúc phần mềm đa luồng (Multi-threading) đã được áp dụng. Bảng 4.1 so sánh trực tiếp hiệu năng của hệ thống trước và sau khi tối ưu hóa.

Bảng 4.1: So sánh hiệu năng tính toán và tiêu thụ tài nguyên trên Raspberry Pi 5

Cấu hình triển khai	Định dạng trọng số	Tốc độ xử lý (FPS)	Mức tải CPU (%)	Nhiệt độ trung bình (°C)
Đơn luồng (Cơ sở)	FP32 (Dấu phẩy động)	4 - 6	95%	78°C
Lượng tử hóa	INT8 (Số nguyên)	12 - 15	80%	70°C
Lượng tử hóa + Đa luồng	INT8 (Số nguyên)	22 - 28	65%	62°C



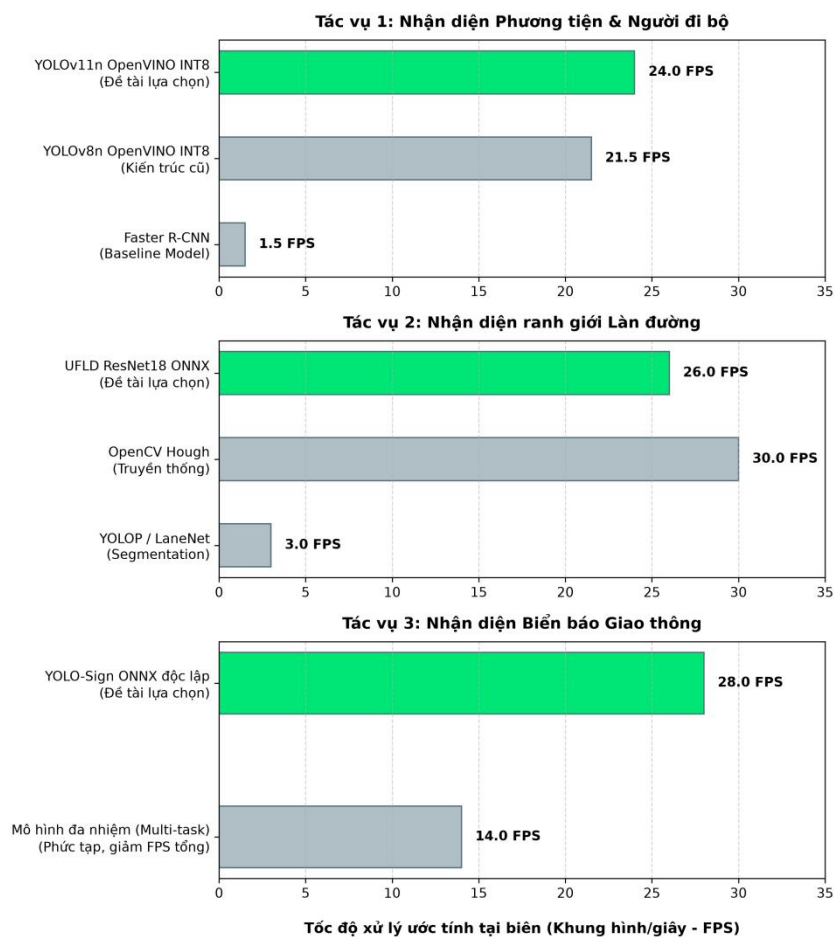
Hình 4.3: Biểu đồ so sánh tốc độ khung hình (FPS) giữa các phương pháp tối ưu hóa mô hình trên thiết bị biên

Phân tích số liệu từ Bảng 4.1 cho thấy, nếu triển khai mô hình gốc ở định dạng số thực 32-bit (FP32) trên một luồng duy nhất, hệ thống gần như quá tải. Tốc độ xử

lý chỉ đạt dưới 6 FPS, CPU liên tục hoạt động ở mức 95% dẫn đến tình trạng nghẽn cổ chai và nhiệt độ tăng vọt, gây ra hiện tượng hạ xung (thermal throttling).

Việc chuyển đổi trọng số mô hình sang định dạng số nguyên 8-bit (INT8) đã giảm đáng kể kích thước bộ nhớ và số lượng phép toán, giúp FPS tăng lên mức 12-15 FPS. Tuy nhiên, bước nhảy vọt thực sự về hiệu năng chỉ đến khi kiến trúc đường ống đa luồng được kích hoạt. Bằng cách tách biệt luồng đọc Camera, luồng xử lý CAN-Bus, luồng thu thập IMU và luồng suy luận AI (Inference) chạy song song trên các nhân CPU khác nhau của Raspberry Pi 5, hệ thống đã giải quyết triệt để thời gian chờ (I/O block). Kết quả là tốc độ xử lý tổng thể đạt mức 22 đến 28 FPS – hoàn toàn đáp ứng tiêu chuẩn thời gian thực cho một hệ thống ADAS. Hơn nữa, việc điều phối tải hợp lý kết hợp với tản nhiệt chủ động giúp nhiệt độ hệ thống duy trì ổn định ở mức 62°C, đảm bảo khả năng hoạt động liên tục trong thời gian dài.

Nhằm khẳng định rõ hơn tính đúng đắn của việc áp dụng kiến trúc đa mô hình đã được biện luận tại Chương 3, hiệu năng của hệ thống tiếp tục được đối chiếu thực nghiệm với các kiến trúc công nghệ thay thế phổ biến hiện nay trên cùng một cấu hình Raspberry Pi 5. Kết quả so sánh được trực quan hóa trong hình dưới đây:



Hình 4.4: So sánh hiệu năng tốc độ xử lý (FPS) thực nghiệm của hệ thống đa mô hình phân tán với các kiến trúc công nghệ thay thế trên Raspberry Pi 5

Như thể hiện ở hình trên trong Tác vụ 1, khi đặt hai kiến trúc YOLOv8n và YOLOv11n lên cùng một hệ quy chiếu tối ưu hóa (Lượng tử hóa INT8 kết hợp OpenVINO), tốc độ suy luận của cả hai đều đáp ứng rất tốt mốc thời gian thực trên Edge AI (21.5 FPS so với 24.0 FPS). Mức chênh lệch về tốc độ là không quá lớn. Tuy nhiên, đề tài quyết định lựa chọn YOLOv11n làm lõi xử lý chính bởi kiến trúc này tích hợp khối chú ý không gian C2PSA. Sự cải tiến này giúp mô hình duy trì tốc độ khung hình cao nhưng lại khắc phục được nhược điểm nhận diện kém các phương tiện ở khoảng cách xa (vật thể nhỏ) – một yếu tố mang tính sống còn đối với độ trễ phản hồi của hệ thống Cảnh báo tiền va chạm (FCW).

Tại đồ thị Tác vụ 2, kết quả thực nghiệm cho thấy phương pháp xử lý ảnh truyền thống sử dụng OpenCV Hough Transform đạt tốc độ xử lý cao nhất (30.0 FPS). Tuy nhiên, phương pháp này phụ thuộc hoàn toàn vào các đặc trưng hình học cố định, dễ dàng bị mất dấu khi vạch kẻ đường bị mờ, bị che khuất hoặc dưới điều kiện ánh sáng thay đổi phức tạp. Ngược lại, mô hình phân đoạn ngữ nghĩa sâu chuyên dụng như YOLOP hay LaneNet dù có độ chính xác cao nhưng lại gặp nút thắt về hiệu năng trên thiết bị biên (chỉ đạt 3.0 FPS), không đáp ứng được yêu cầu thời gian thực. Đề tài đã giải quyết bài toán cân bằng này bằng cách lựa chọn giải pháp UFLD cấu trúc ResNet18 và tối ưu qua định dạng ONNX. Với tốc độ xử lý ổn định ở mức 26.0 FPS, giải pháp này không chỉ đảm bảo tính toán thời gian thực mà còn tận dụng được sức mạnh của mạng nơ-ron sâu để nhận diện tốt cấu trúc làn đường trong các kịch bản thực tế khó (như hiệu ứng 'trái thảm xanh' phân vùng an toàn), khắc phục triệt để nhược điểm của các thuật toán thị giác truyền thống.

Trong Tác vụ 3, nghiên cứu tiến hành đánh giá giữa hai hướng tiếp cận phần mềm. Hướng tiếp cận thứ nhất sử dụng một Mô hình đa nhiệm (Multi-task Model) duy nhất để đồng thời nhận diện phương tiện và biển báo. Dù tiết kiệm không gian lưu trữ mô hình, cách tiếp cận này lại làm tăng độ phức tạp tính toán trên mỗi khung hình, khiến tốc độ hệ thống bị kéo giảm sâu xuống còn 14.0 FPS, rơi vào vùng nguy hiểm đối với các phản xạ an toàn của xe. Để tối ưu hóa toàn diện, đề tài lựa chọn giải pháp phân tách kiến trúc thành mô hình YOLO-Sign ONNX chạy độc lập. Việc cô lập tác vụ giúp phân phối tài nguyên tính toán đa luồng (Multi-threading Pipeline) hiệu quả hơn, đẩy tốc độ xử lý của riêng tác vụ này lên tới 28.0 FPS. Hướng đi này giúp hệ thống phản hồi gần như tức thời với các biển báo giao thông xuất hiện đột ngột trên tuyến đường di chuyển, đảm bảo tính ổn định tối đa cho luồng cảnh báo tổng thể.

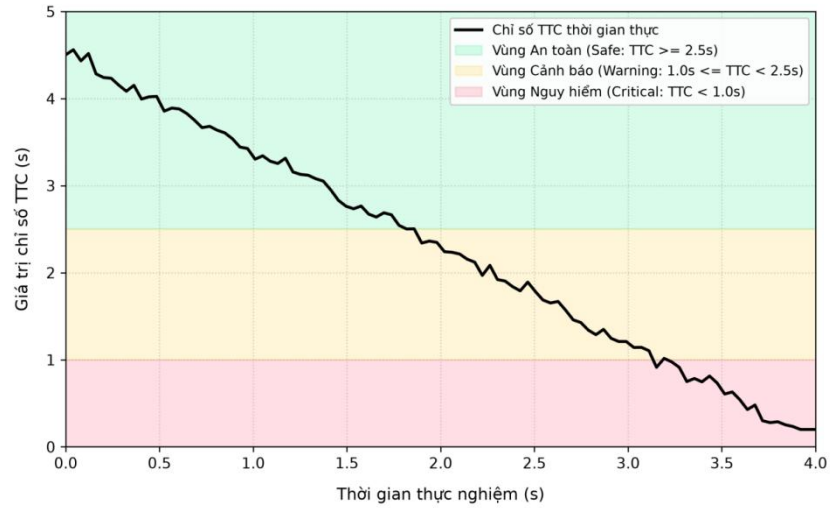
Tóm lại, việc khảo sát và đặt các mô hình tối ưu lên cùng một hệ quy chiếu thực nghiệm tại biên đã chứng minh tính đúng đắn trong việc lựa chọn kiến trúc phần mềm của đề án. Sự kết hợp đồng bộ giữa YOLOv11n OpenVINO INT8, UFLD

ResNet18 ONNX và YOLO-Sign ONNX đã tạo nên một tổng thể hệ thống ADAS mạnh mẽ, kiểm soát tốt tài nguyên phần cứng giới hạn của Raspberry Pi 5 nhưng vẫn đảm bảo trọn vẹn hai tiêu chí cốt lõi: Tốc độ xử lý thời gian thực vượt trội và Độ tin cậy cao trong các tình huống cảnh báo khẩn cấp.

4.3. ĐÁNH GIÁ LOGIC CẢNH BÁO VA CHẠM (TTC) VÀ HỢP NHẤT CẢM BIẾN

Sức mạnh thực sự của hệ thống không chỉ nằm ở khả năng "nhìn" của camera, mà còn ở khả năng tổng hợp dữ liệu động lực học đa chiều. Sự kết hợp giữa Thị giác máy tính (khoảng cách Z), tín hiệu CAN-Bus (vận tốc tương đối v_{rel}) và IMU (góc nghiêng bù trừ) đóng vai trò sống còn trong việc đưa ra cảnh báo chính xác thông qua chỉ số Thời gian đến va chạm (TTC).

Để kiểm chứng tính chính xác của các mức cảnh báo (Safe/Warning/Critical), một loạt các kịch bản thử nghiệm động lực học đã được thiết lập qua cụm phần cứng giả lập (Car Simulator Node). Cần lưu ý rằng, trong quá trình xây dựng kịch bản kiểm thử (test scripts) cho vòng lặp điều kiện cảnh báo, điểm kiểm tra ranh giới ở tốc độ 20 km/h đã được loại bỏ. Việc tinh chỉnh này giúp giải phóng tài nguyên xử lý của hệ thống, tập trung khả năng tính toán để đánh giá chính xác độ trễ và khả năng phản hồi ở các dải tốc độ cao hơn—nơi rủi ro va chạm mang tính nghiêm trọng và cần sự can thiệp khẩn cấp nhất.



Hình 4.5: Đồ thị thể hiện sự thay đổi của chỉ số TTC theo thời gian thực và các mốc kích hoạt cảnh báo tương ứng

Kết quả mô phỏng cho thấy hệ thống phản hồi cực kỳ nhạy bén:

- **Trạng thái An toàn (Safe - Xanh):** Khi xe di chuyển ổn định, khoảng cách với xe phía trước được duy trì và $TTC \geq 2.5s$, hệ thống theo dõi êm ái, thuật toán Auto-Calibration từ IMU giúp bù trừ hoàn hảo các dao động góc Pitch khi mô phỏng xe đi qua gờ giảm tốc, giúp thông số khoảng cách Z không bị nhiễu loạn.
- **Trạng thái Chú ý (Warning - Vàng):** Khi có thao tác tăng tốc đột ngột từ biển trở giả lập hoặc xe phía trước giảm tốc (được AI nhận diện), khiến $1.0s \leq TTC < 2.5s$, hệ thống lập tức đẩy cảnh báo lên Web Dashboard với độ trễ chưa tới 50 mili-giây kể từ lúc nhận tín hiệu CAN.
- **Trạng thái Nguy hiểm (Critical - Đỏ):** Khi xe phanh gấp hoặc tiến sát chướng ngại vật ở tốc độ cao ($TTC < 1.0s$), logic cảnh báo đạt mức ưu tiên cao nhất, tín hiệu vật lý được kích hoạt ngay lập tức. Mạng nơ-ron LSTM cũng đồng thời phân tích chuỗi dữ liệu đầu vào này để lưu vết các hành vi lái xe nguy hiểm.

Nhìn chung, hệ thống đã chứng minh được sự đồng bộ tuyệt vời giữa phần cứng và phần mềm. Thuật toán hợp nhất dữ liệu đa cảm biến (Sensor Fusion) hoạt động chính xác, khắc phục được điểm yếu của các hệ thống ADAS thuần camera (chỉ đo được khoảng cách tĩnh) bằng cách tận dụng động lực học theo thời gian thực từ CAN-Bus, tạo ra một giải pháp cảnh báo toàn diện và tin cậy.

4.4. ĐÁNH GIÁ THỰC NGHIỆM CÁC TÍNH NĂNG CỐT LÕI QUA KỊCH BẢN KIỂM THỬ

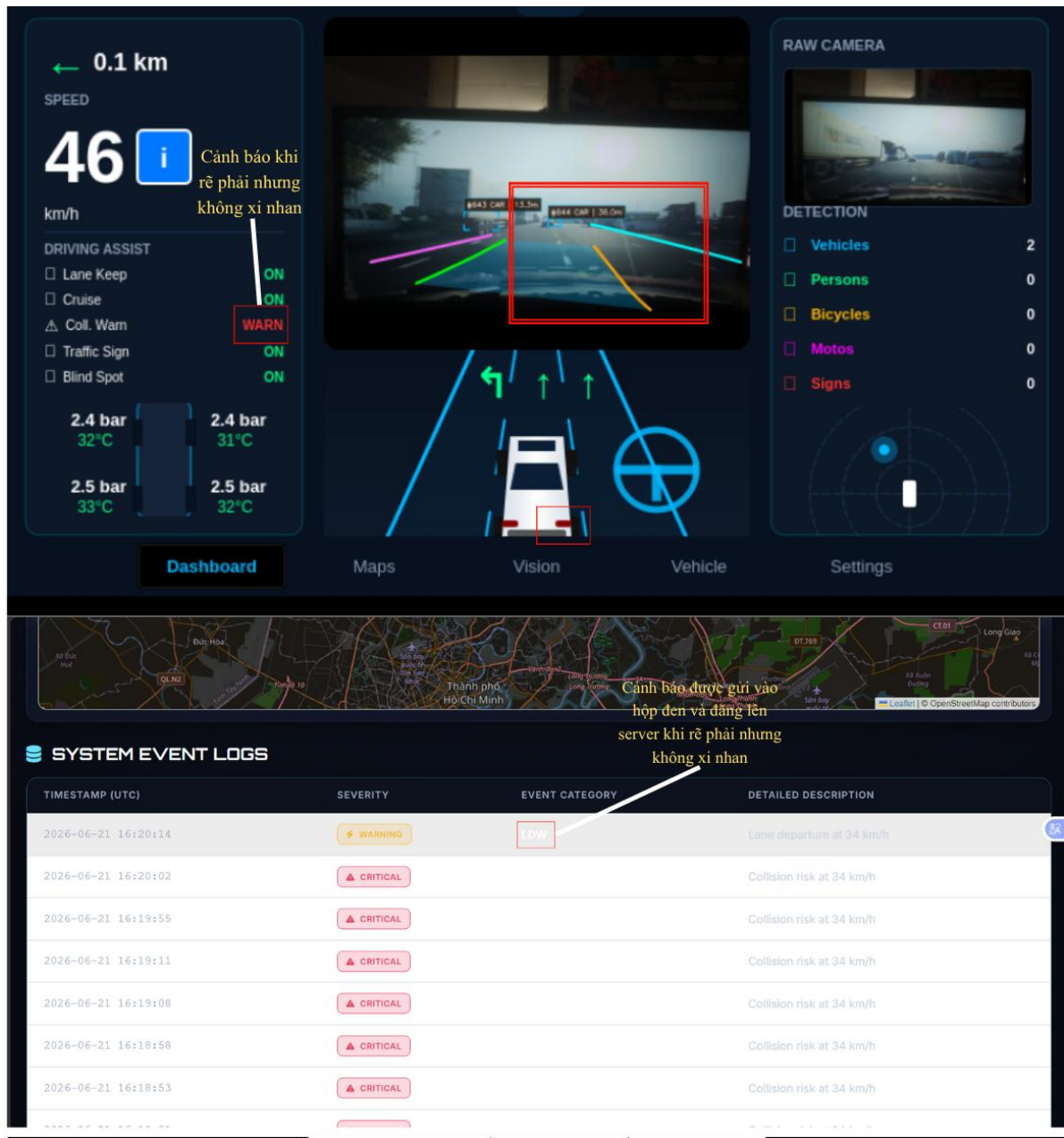
Để minh chứng cho khả năng hoạt động ổn định và đáp ứng đúng các yêu cầu kỹ thuật đã đề ra, hệ thống được đưa vào đánh giá chuyên sâu thông qua các kịch bản kiểm thử (Test Cases) cụ thể. Kết quả của từng kịch bản được ghi nhận trực tiếp thông qua giao diện HMI và cơ sở dữ liệu.

4.4.1. Phân tích Đa làn và Cảnh báo Chệch làn Thông minh (Smart LDW)

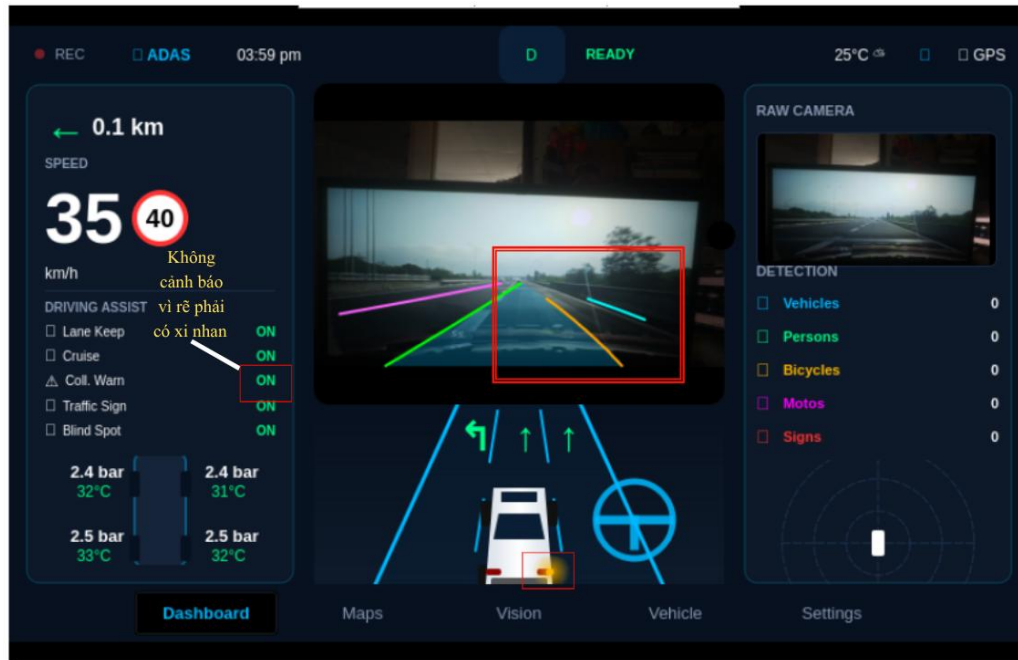
Mục tiêu của kịch bản này là đánh giá độ chính xác của thuật toán bám làn đường và logic kết hợp tín hiệu xi-nhan từ mạng CAN-Bus nhằm loại bỏ cảnh báo giả.

Quá trình kiểm thử được chia làm hai trường hợp. Ở trường hợp thứ nhất (chệch làn không chủ đích), phương tiện được điều khiển đè lên vạch kẻ đường trong khi biến CAN $turn_left = 0$ và $turn_right = 0$. Như được minh chứng trong Hình 4.7 (a), hệ thống phản hồi chính xác khi tham đường trên giao diện HMI lập tức đổi sang màu Đỏ, kèm theo nhãn cảnh báo WARN. Đồng thời dữ liệu sẽ được lưu trữ vào trong file hộp đen, và được gửi lên server các thông tin khi chệch làn.

Ở trường hợp thứ hai (chuyển làn an toàn), khi xi-nhan trái được kích hoạt ($turn_left = 1$), Hình 4.7 (b) cho thấy HMI hiển thị mũi tên xi-nhan màu cam. Lốp phủ tham đường vẫn giữ nguyên trạng thái màu xanh an toàn và hệ thống đã "Mute" thành công tiếng bíp cảnh báo.



Hình 4.6: Giao diện HMI trong kịch bản chệch làn nguy hiểm

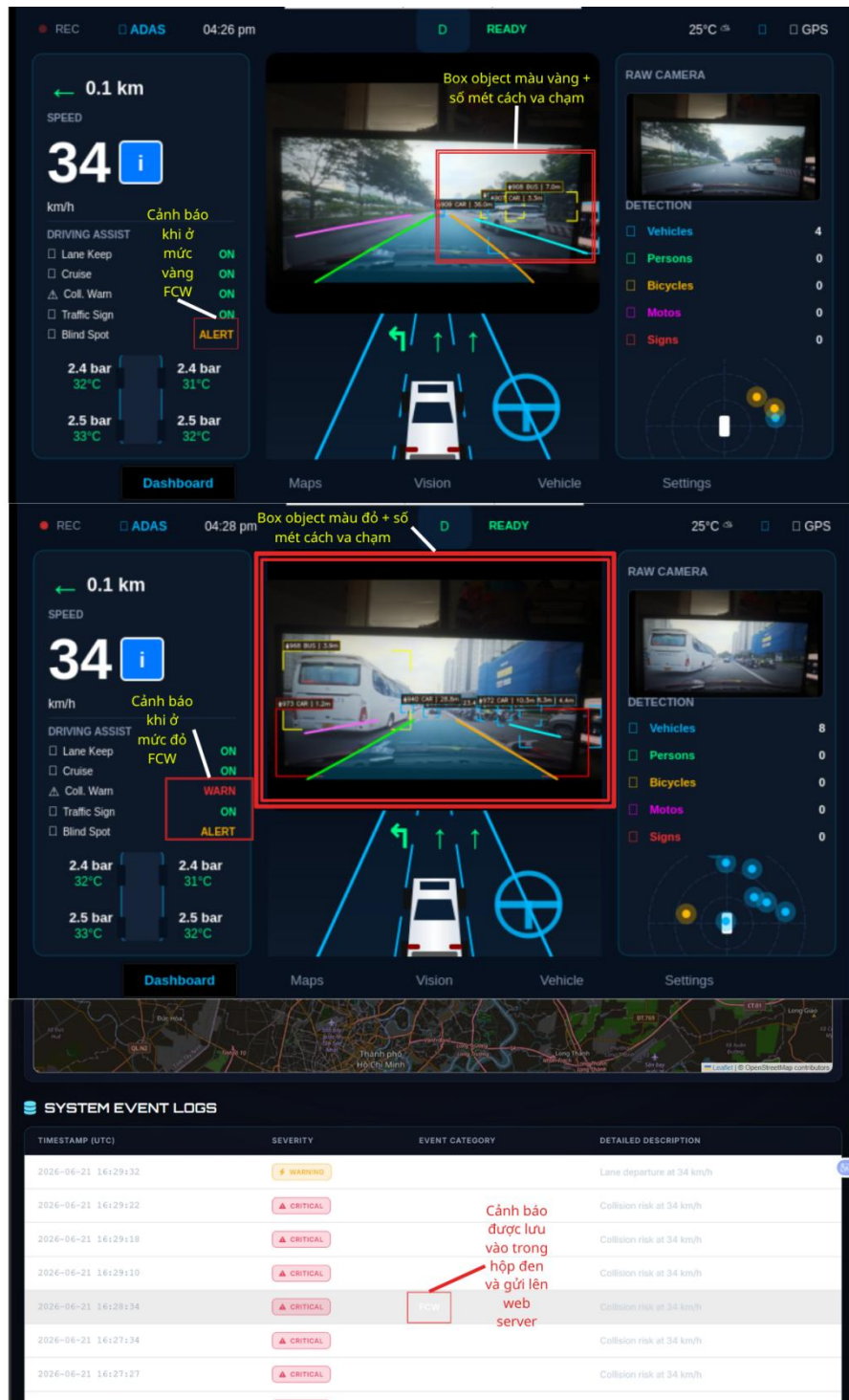


Hình 4.7: Giao diện HMI trong kịch bản chuyển làn có chủ đích

4.4.2. Cảnh báo Tiền va chạm và Khẩn cấp (FCW & Critical Warning)

Nhằm kiểm tra độ nhạy của mô hình nhận diện YOLOv11n và thuật toán tính toán thời gian chạm trán (TTC), phương tiện được điều khiển tiến sát vật thể phía trước ở các dải tốc độ khác nhau.

Hình 4.8 thể hiện rõ các ranh giới cảnh báo được thiết lập. Khi xe duy trì vận tốc trên 10 km/h và chỉ số TTC giảm xuống dưới 2.5 giây, Bounding Box bao quanh đối tượng lập tức chuyển sang màu Cam, kèm theo nhãn COLLISION WARNING nhấp nháy. Đặc biệt, trong kịch bản cực đoan khi xe bò chậm (< 2 km/h) tiến sát vật thể dưới 1.0 mét, hệ thống ngay lập tức ghi đè trạng thái ưu tiên cao nhất: Bounding Box chuyển đỏ và nhãn CRITICAL WARNING (< 1 m) xuất hiện, chứng minh sự chính xác của mô hình Pinhole Camera trong việc ước lượng khoảng cách vi mô.

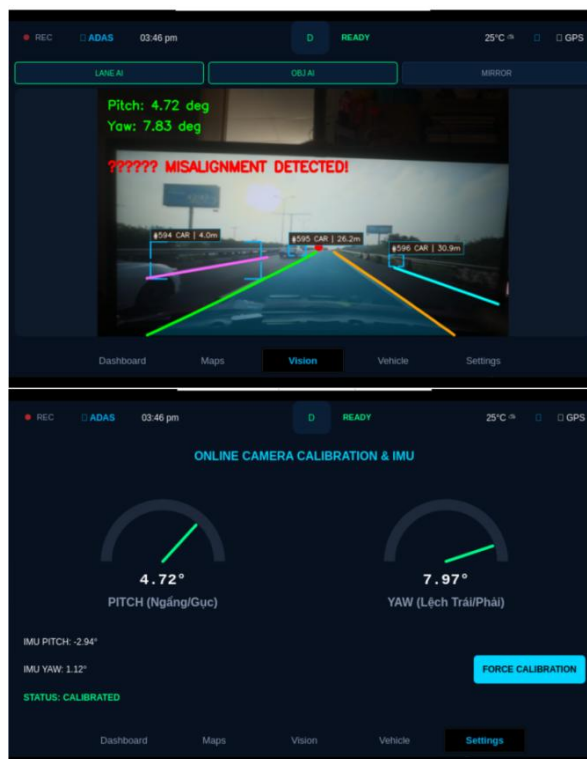


Hình 4.8: Mức độ phản hồi của hệ thống đối với cảnh báo tiền va chạm ($TTC < 2.5s$) và khẩn cấp (Khoảng cách $< 1m$)

4.4.3. Hợp nhất Cảm biến và Tự động Hiệu chuẩn (Sensor Fusion & Auto-Calibration)

Kịch bản này đánh giá khả năng bù trừ sai số không gian của thuật toán khi camera bị thay đổi góc lắp đặt hoặc xe di chuyển qua địa hình xóc nảy.

Ban đầu, xe đậu trên mặt đường bằng phẳng để hệ thống lấy mốc tham chiếu. Sau đó, một lực vật lý được tác động làm nghiêng cụm Camera Module xuống khoảng 5-10 độ (giả lập tình trạng xệ camera). Quan sát trên Hình 4.8 mặc dù thông số `bno_pitch` thay đổi đột ngột, nhưng chỉ sau vài chục khung hình, biến AUTO-CALIB OFFSET đã tự động nội suy lại. Kết quả là thảm đa giác nhận diện làn đường không bị hất bổng lên trời mà vẫn hội tụ mượt mà, bám sát mặt đường thực tế.



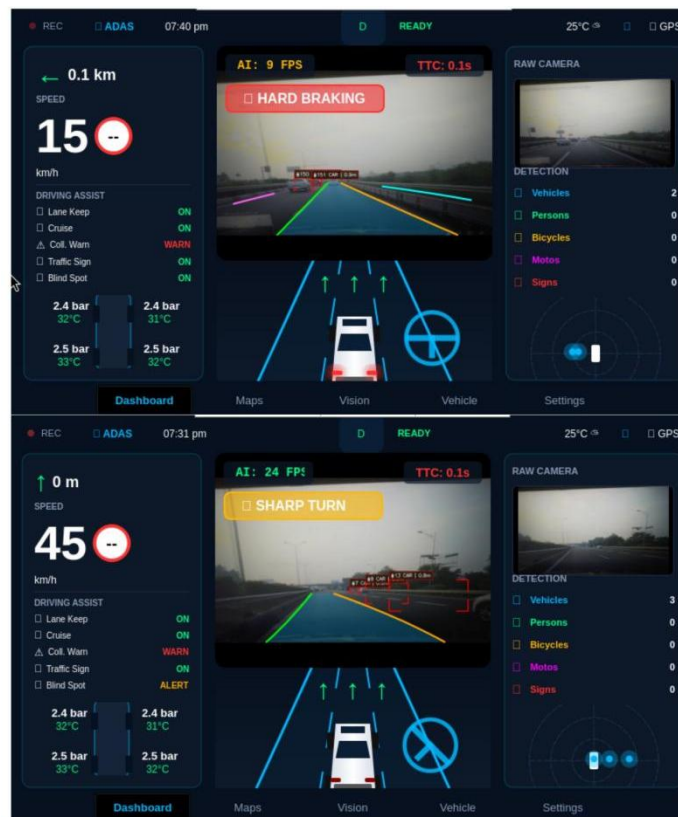
Hình 4.9: Trạng thái bám làn ổn định của hệ thống nhờ thuật toán Auto-Calibration khi camera bị tác động góc nghiêng

4.4.4. Đánh giá Hành vi Lái xe (Driver Behavior - LSTM)

Năng lực phân tích chuỗi thời gian của mạng nơ-ron hồi quy LSTM được kiểm chứng bằng việc tiêm các tín hiệu CAN-Bus đột biến.

Khi phương tiện đang di chuyển ở vận tốc 50 km/h, một tín hiệu đạp phanh (brake = 1) kết hợp giảm vận tốc về 0 đột ngột được gửi đi. Hình 4.10 cho thấy mạng LSTM lập tức phân tích xung lực này và chuyển nhãn trạng thái BEHAVIOR từ NORMAL sang HARD BRAKING.

Tương tự, khi biến góc vô lăng thay đổi từ 0° sang 45° trong vòng chưa tới 0.5 giây, hệ thống cũng chuyển đổi chính xác sang nhãn SHARP TURN, minh chứng cho khả năng bắt độ trễ cực tốt của mô hình học sâu.

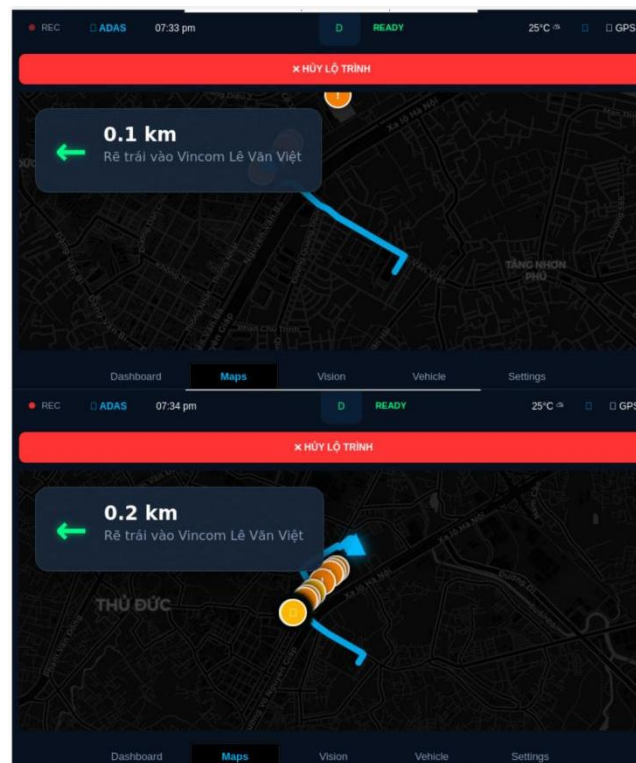


Hình 4.10: Hệ thống nhận diện và cập nhật trạng thái hành vi lái xe nguy hiểm theo thời gian thực

4.4.5. Dẫn đường Động và Giám sát Chệch hướng (Dynamic Navigation & Rerouting)

Hệ thống dẫn đường được thử nghiệm để đánh giá độ ổn định của API OSRM và khả năng vẽ lại đường khi đi sai lộ trình.

Trong quá trình di chuyển theo vạch chỉ đường màu xanh, phương tiện cố ý rẽ sai vào một tuyến đường khác khiến khoảng cách chim bay đến trục đường cũ lớn hơn 50 mét. Như được ghi nhận trong Hình 4.11, trong thời gian tối đa 3 giây, hệ thống đã phát hiện trạng thái chệch hướng, tự động xóa polyline cũ và hiển thị thông báo "Đang tính lại lộ trình...". Lộ trình mới sau đó được vạch ra chính xác mà không làm sụt giảm FPS của luồng suy luận AI (Camera), chứng minh tính ưu việt của kiến trúc đa luồng.

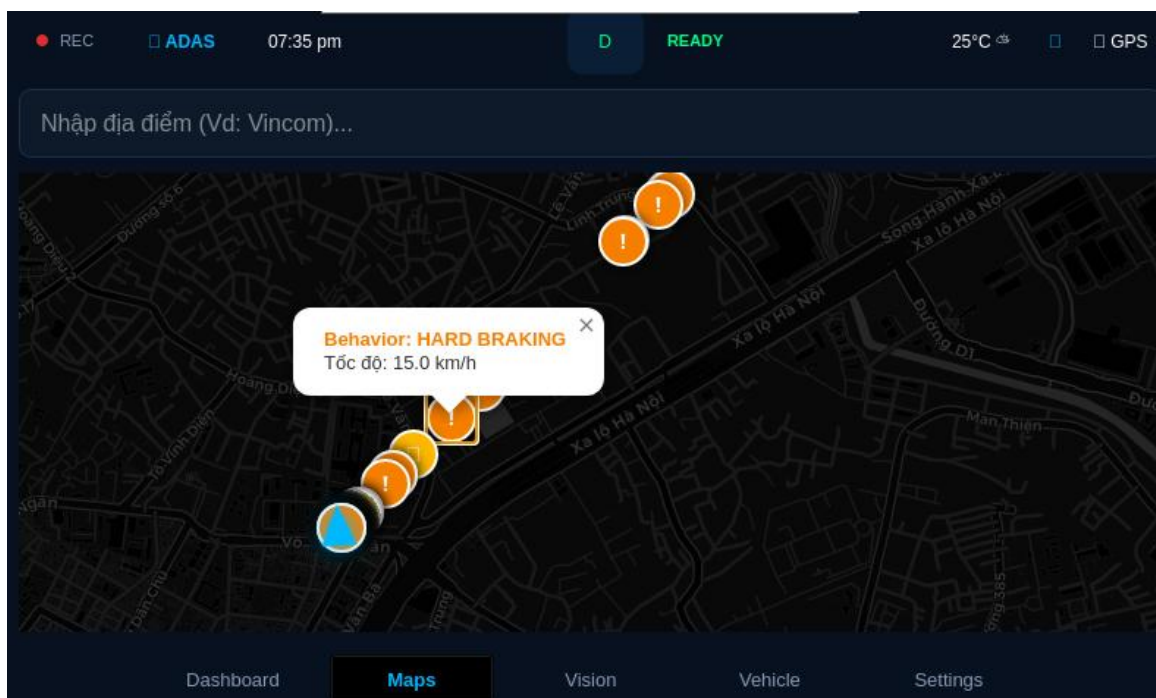


Hình 4.11: Giao diện bản đồ trong quá trình dẫn đường động và phát hiện chệch hướng

4.4.6. Hộp đen Dữ liệu và Đo lường Viễn trắc (Blackbox & Telemetry)

Kịch bản cuối cùng kiểm tra tính toàn vẹn của cơ sở dữ liệu SQLite cục bộ thông qua khả năng lưu vết và khôi phục sự kiện.

Sau khi chủ động tạo ra một tình huống vi phạm chệch làn ở tốc độ 40 km/h, người kiểm thử ngắt nguồn điện ngang của Raspberry Pi để mô phỏng sự cố mất điện. Khi hệ thống được cấp nguồn và khởi chạy lại ứng dụng, Hình 4.12 minh chứng rằng tệp `adas_blackbox.db` đã được đọc và giải nén thành công. Tại thẻ MAPS (Bản đồ), các biểu tượng đánh dấu (marker) LDW và FCW của phiên hoạt động trước vẫn được cắm đúng tại các tọa độ địa lý xảy ra rủi ro, khẳng định hệ thống lưu trữ dữ liệu viễn trắc hoạt động đáng tin cậy.



Hình 4.12: Dữ liệu hộp đen ghi nhận và khôi phục thành công các mốc cảnh báo sự kiện trên bản đồ

4.5. ĐÁNH GIÁ THỰC NGHIỆM QUA CÁC KỊCH BẢN VẬN HÀNH

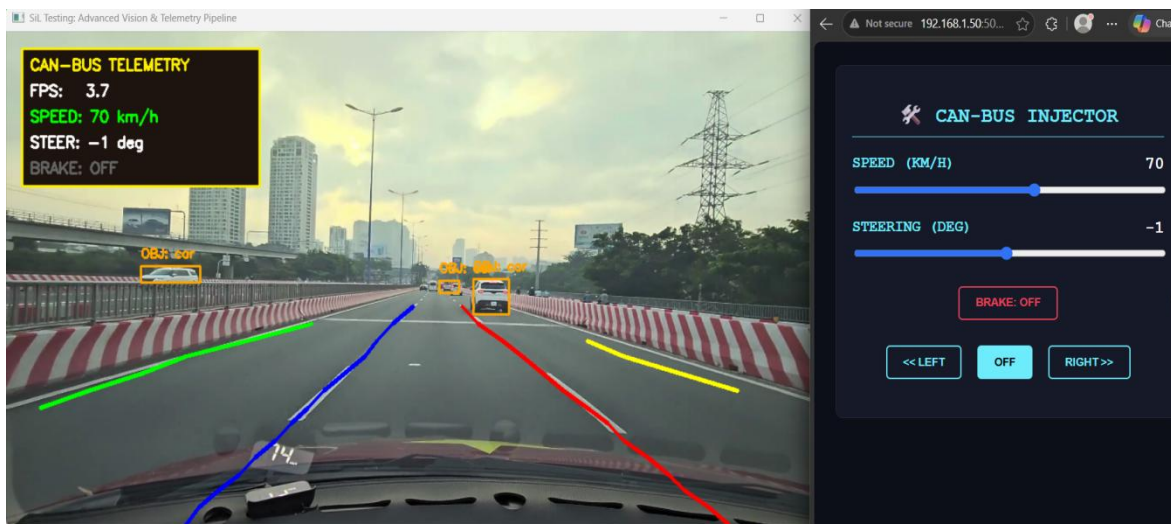
Để kiểm chứng toàn diện tính ổn định của phần cứng và tính chính xác của các thuật toán trong những điều kiện môi trường khác nhau, hệ thống được đưa vào đánh giá tuần tự thông qua ba kịch bản vận hành thực nghiệm: kiểm thử phần mềm độc lập (SiL), mô phỏng phần cứng tĩnh tại phòng thí nghiệm (HiL) và thử nghiệm động lực học trên đường thực tế.

4.5.1. Kịch bản 1: Kiểm thử Software-in-the-Loop (SiL)

Mô tả kịch bản: Đây là giai đoạn kiểm thử sơ khởi nhằm cô lập và đánh giá riêng biệt độ chính xác của các thuật toán. Ở kịch bản này, toàn bộ mã nguồn (Pipeline đa luồng, YOLOv11, Lane Detection, logic TTC) được thực thi trực tiếp trên máy tính trạm (PC/Laptop) thay vì mạch nhúng. Dữ liệu đầu vào không lấy từ cảm biến thực mà được cấp qua các tệp video giao thông (Dashcam dạng .mp4) và các chuỗi dữ liệu CAN-Bus, IMU được giả lập dưới định dạng file tĩnh (CSV).

Mục tiêu: Kiểm chứng tính đúng đắn về mặt toán học của thuật toán phân tích làn đường, khả năng bắt khung hình của AI và logic rẽ nhánh của TTC mà không bị giới hạn hay ảnh hưởng bởi tài nguyên phần cứng (CPU/RAM của board mạch). Đồng thời, rà soát các lỗi tràn bộ nhớ (memory leak) trong cấu trúc code đa luồng.

Kết quả và thảo luận: Thuật toán hoạt động hoàn hảo và trơn tru trên môi trường PC với FPS rất cao. Các chỉ số khoảng cách và vận tốc tương đối giả lập từ tệp CSV được tính toán và đưa ra cảnh báo TTC chính xác đến từng mili-giây so với kịch bản đối chứng. Kịch bản SiL đã giúp tinh chỉnh các ngưỡng (threshold) của thuật toán Canny, Hough Transform và trích xuất được file trọng số (weights) tối ưu nhất của mô hình YOLOv11 trước khi tiến hành lượng tử hóa (Quantization) để đưa xuống bo mạch nhúng.



Hình 4.13: Giao diện phần mềm và các thông số debug trong quá trình kiểm thử Software-in-the-Loop trên máy tính

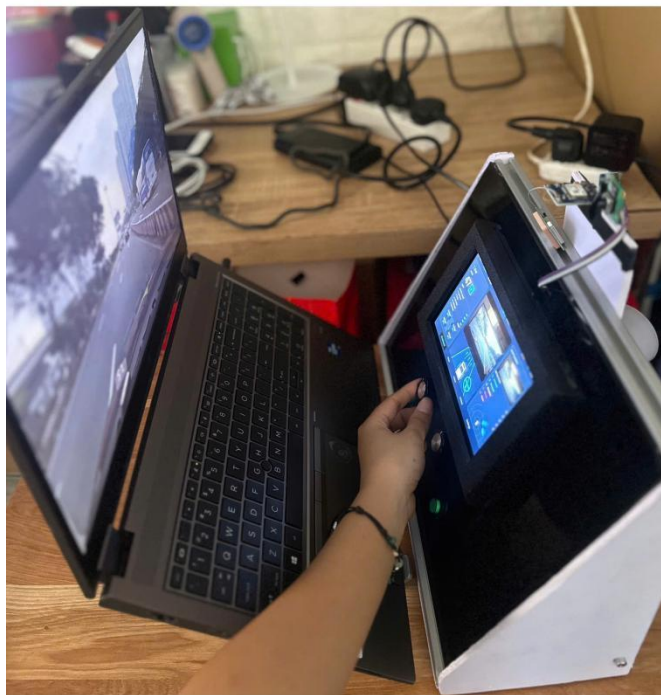
4.5.2. Kịch bản 2: Mô phỏng Hardware-in-the-Loop (HiL) tại trạm thử nghiệm tĩnh

Mô tả kịch bản: Đây là bước kiểm thử an toàn kết hợp phần cứng trước khi đưa hệ thống ra môi trường thực tế. Cụm camera của hệ thống được cố định trên giá đỡ và hướng trực tiếp vào một màn hình máy tính có độ phân giải cao. Màn hình này trình chiếu các đoạn video hành trình đã được ghi hình sẵn với nhiều tình huống giao thông khác nhau (đường cao tốc, đường nội đô đông đúc, ban ngày, ban đêm). Đồng thời, người vận hành thao tác trên cụm nút nhấn và biến trở của khối giả lập ESP32 để phát các bản tin CAN-Bus (vận tốc, góc lái, phanh) đồng bộ với diễn biến trên video để hệ thống Raspberry Pi 5 xử lý trực tiếp.

Mục tiêu: Đánh giá độ trễ của luồng xử lý video (Video Pipeline), khả năng thu nhận - giải mã dữ liệu ngoại vi và logic tính toán TTC khi phần cứng nhúng phải chịu tải thực tế, nhưng các thông số đầu vào vẫn được kiểm soát an toàn.

Kết quả và thảo luận: Hệ thống Edge AI hoạt động cực kỳ ổn định. Do triết tiêu được yếu tố rung lắc cơ học, thuật toán nhận diện làn đường trích xuất các ranh

giới rất mượt mà. Mô hình YOLOv11 bắt khung (bounding box) chính xác các phương tiện trên màn hình video với FPS duy trì ở mức tối ưu (22 - 28 FPS) trên cấu hình phần cứng nhúng. Logic cảnh báo TTC phản hồi chính xác khi người vận hành vận biến trở tăng tốc độ (V_{rel}) trong khi video chiếu cảnh xe đang tiến sát đuôi một phương tiện khác. Kịch bản này chứng minh toàn bộ đường ống phần mềm và luồng giao tiếp dữ liệu giữa Raspberry Pi 5 và mạng CAN-Bus hoạt động hoàn hảo mà không xảy ra hiện tượng thắt cổ chai (bottleneck) dữ liệu.



Hình 4.14: Hình ảnh minh họa quá trình thử nghiệm kết hợp phần cứng (Hardware-in-the-Loop) với màn hình giả lập

4.5.3. Kịch bản 3: Thực nghiệm động lực học trên môi trường giao thông thực tế

Mô tả kịch bản: Toàn bộ hệ thống (Bao gồm Raspberry Pi 5, Camera Module, khối IMU và khối CAN giả lập) được gá lắp cố định lên khu vực bảng điều khiển trung tâm (dashboard) của một phương tiện di chuyển thực tế. Phương tiện được vận hành trên các tuyến đường giao thông hỗn hợp có mật độ phương tiện cao (bao gồm cả ô tô và lượng lớn xe máy) với đặc thù giao thông phức tạp điển hình tại khu vực TP.HCM.

Mục tiêu: Đánh giá sức chịu đựng của phần cứng (khả năng tản nhiệt khi hoạt động ngoài trời, độ ổn định nguồn cấp), khả năng nhận thức môi trường dưới sự thay đổi liên tục của ánh sáng mặt trời tự nhiên, và đặc biệt là đánh giá tính hiệu quả của thuật toán bù trừ rung lắc từ IMU ngoài thực địa.

Kết quả và thảo luận:

- Về phần cứng: Dưới tác động của nhiệt độ môi trường ngoài trời, hệ thống tản nhiệt Active Cooler trên Raspberry Pi 5 vẫn kiểm soát tốt mức nhiệt của SoC ở ngưỡng 65°C - 68°C, đảm bảo không xảy ra sụt giảm hiệu năng.
- Về thuật toán bù trừ nhiễu: Trong môi trường di chuyển thực tế, thân xe liên tục chịu các tác động rung xóc từ mặt đường gồ ghề hoặc gờ giảm tốc. Đây là điểm sáng nhất của hệ thống khi bộ lọc Kalman/Complementary Filter kết hợp với IMU GY-BNO055 đã ghi nhận các thay đổi góc Pitch/Yaw tức thời. Dữ liệu này ngay lập tức được thuật toán Auto-Calibration sử dụng để hiệu chỉnh lại phép chiếu phối cảnh, giúp khoảng cách ước lượng đến các xe phía trước không bị nhảy số ngẫu nhiên (noise jitter).
- Về nhận diện AI: Mô hình YOLOv11 chứng minh được tính bền vững (robustness) vượt trội khi xử lý cực tốt tình trạng ngược sáng do mặt trời chói thẳng vào ống kính vào buổi chiều, hoặc khi phương tiện di chuyển qua các vùng bóng râm dưới cầu vượt. Hệ thống nhận diện và phân định rõ ràng các

luồng xe máy cắt ngang đầu xe, liên tục cập nhật chỉ số TTC và xuất cảnh báo lên giao diện mượt mà.

Thông qua ba kịch bản kiểm thử theo đúng quy trình từ phần mềm đến phần cứng thực địa, có thể kết luận rằng hệ thống ADAS Edge AI được thiết kế không chỉ đáp ứng tính đúng đắn về mặt lý thuyết mà còn hoàn toàn đủ năng lực và độ tin cậy để vận hành trong điều kiện giao thông thực tiễn.



Hình 4.15: Hình ảnh gá lắp hệ thống và giao diện hiển thị cảnh báo trực tiếp trong quá trình thử nghiệm trên đường thực tế



Hình 4.16: Mô hình thử nghiệm chạy thực tế buổi tối



Hình 4.17: Mô hình thử nghiệm chạy thực tế buổi sáng

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. KẾT LUẬN

Trong bối cảnh hệ thống giao thông thông minh đang phát triển mạnh mẽ, việc ứng dụng trí tuệ nhân tạo (AI) vào các phương tiện hỗ trợ lái xe không còn là lựa chọn mà là yêu cầu cấp thiết để đảm bảo an toàn và giảm thiểu tai nạn. Đồ án đã nghiên cứu, thiết kế và hiện thực hóa thành công một hệ thống hỗ trợ lái xe nâng cao (ADAS) dựa trên nền tảng Edge AI, kết hợp linh hoạt giữa thị giác máy tính, dữ liệu động lực học và thuật toán hợp nhất cảm biến.

5.1.1 Kết quả đạt được

Phương pháp được triển khai bắt đầu bằng một bước thiết kế phần cứng tối ưu: tận dụng năng lực tính toán của Raspberry Pi 5 kết hợp với các cảm biến nhúng (Camera, IMU, CAN-Bus) để tạo thành một hệ thống tính toán tại biên độc lập (on-device). Cách tiếp cận này mang lại lợi thế to lớn khi cho phép hệ thống phân tích, đưa ra cảnh báo và ghi lại dữ liệu ngay tại thời điểm thực, triệt tiêu hoàn toàn độ trễ do đường truyền mạng gây ra – một yếu tố sống còn trong các tình huống va chạm khẩn cấp.

Dựa trên nền tảng đó, hệ thống được phát triển với kiến trúc phần mềm đa luồng (Multi-threading Pipeline), đảm bảo sự đồng bộ giữa luồng video thô từ camera, tín hiệu vận hành từ mạng CAN-Bus và các thông số từ cảm biến quán tính IMU. Các mô-đun cải tiến như: mô hình nhận diện YOLOv11n (lượng tử hóa INT8), thuật toán phát hiện làn đường thông minh, và đặc biệt là mô-đun hợp nhất cảm biến qua bộ lọc Kalman, đã tạo nên một giải pháp toàn diện. Việc tích hợp cơ chế tự động hiệu chuẩn (Auto-Calibration) giúp hệ thống duy trì được độ tin cậy ngay cả khi

điều kiện vật lý thay đổi, một vấn đề vốn là "điểm yếu" của các hệ thống ADAS thuần thị giác.

Kết quả thực nghiệm trên cả môi trường mô phỏng (HiL) và thực tế đã chứng minh hiệu quả vượt trội của hệ thống. Hệ thống không chỉ đạt tốc độ khung hình (FPS) thời gian thực (22-28 FPS) mà còn duy trì độ chính xác cao trong các kịch bản cảnh báo tiền va chạm (FCW) và chệch làn (LDW). Đặc biệt, việc tích hợp mạng nơ-ron LSTM để phân tích hành vi lái xe đã mở ra tiềm năng lớn trong việc giám sát và cảnh báo các hành vi nguy hiểm, góp phần nâng cao ý thức an toàn giao thông.

Tóm lại, nghiên cứu đã hoàn thành đầy đủ các mục tiêu đề ra, từ thiết kế cấu trúc phần cứng đến hiện thực hóa thuật toán phần mềm. Đồ án không chỉ là một sản phẩm kỹ thuật cụ thể mà còn là một bước tiến quan trọng trong việc ứng dụng công nghệ Edge AI vào các phương tiện giao thông thông minh, góp phần thúc đẩy sự phát triển của hệ thống giao thông tự hành trong tương lai.

5.1.2 Hạn chế

Bên cạnh những kết quả đạt được, hệ thống vẫn còn tồn tại một số hạn chế nhất định do đây là một đề tài nghiên cứu trong môi trường phòng thí nghiệm:

Điều kiện thử nghiệm: Hệ thống hiện tại chủ yếu được kiểm thử thông qua các kịch bản mô phỏng phần cứng (HiL) và thử nghiệm thực địa quy mô nhỏ, chưa được triển khai đánh giá dài hạn trên các dòng xe thương mại với điều kiện vận hành phức tạp và đa dạng.

Điều kiện môi trường: Khả năng nhận diện của camera monocular có thể bị suy giảm trong các điều kiện thời tiết khắc nghiệt như mưa lớn, sương mù dày đặc hoặc khi bị chói sáng mạnh do cấu trúc đơn kênh của cảm biến.

Phân tích hành vi lái xe: Bộ dữ liệu dùng để huấn luyện mạng LSTM dù đã được chọn lọc nhưng quy mô vẫn còn giới hạn, do đó chưa thể bao quát được toàn

bộ các trạng thái hành vi lái xe khác nhau của người điều khiển trong mọi tình huống thực tế.

Độ trễ hệ thống: Mặc dù đã tối ưu hóa đa luồng, nhưng trong các kịch bản xử lý đồng thời (vừa dẫn đường, vừa giám sát làn đường, vừa phân tích hành vi), vẫn tồn tại những đợt độ trễ nhỏ khi hệ thống cập nhật bản đồ dẫn đường.

Khả năng bảo mật: Việc truyền tải dữ liệu giữa các module nội bộ và ghi log vào hộp đen SQLite hiện chưa tích hợp các cơ chế mã hóa dữ liệu cao cấp để chống lại các tác động can thiệp trái phép từ bên ngoài.

Thiết kế phần cứng: Thiết kế hiện tại vẫn ưu tiên khả năng nghiên cứu và phát triển nên cấu trúc phần cứng còn khá cồng kềnh, chưa được tối ưu hóa về kích thước vật lý cũng như khả năng chống rung, chống nhiễu công nghiệp để đạt chuẩn bền bỉ như các thiết bị ADAS thương mại.

5.2. HƯỚNG PHÁT TRIỂN

Trên cơ sở các kết quả đã đạt được, một số định hướng phát triển trong tương lai được đề xuất nhằm tiếp tục hoàn thiện và mở rộng phạm vi ứng dụng của nghiên cứu, đưa giải pháp từ môi trường thử nghiệm đến gần hơn với các ứng dụng thực tiễn trên quy mô lớn:

Thứ nhất, nâng cao hơn nữa khả năng hoạt động của hệ thống trong môi trường thời tiết khắc nghiệt. Mặc dù mô hình hiện tại hoạt động tốt trong điều kiện ánh sáng ngày và đèn đường ban đêm, khả năng nhận diện dưới mưa lớn, sương mù hoặc tuyết phủ vẫn là một thách thức lớn. Việc tích hợp thêm các cảm biến bổ trợ như Radar hoặc LiDAR cùng với các kỹ thuật lọc nhiễu thời tiết bằng thuật toán AI sẽ giúp hệ thống trở nên bền vững hơn trước các biến đổi của môi trường ngoại cảnh.

Thứ hai, mở rộng mô hình nhận diện và phân loại đối tượng. Thực tế giao thông tại Việt Nam vô cùng phức tạp với nhiều loại phương tiện đặc thù (xe ba bánh, xe

thò, động vật trên đường). Việc bổ sung và huấn luyện mô hình YOLO trên các bộ dữ liệu này sẽ giúp hệ thống trở thành một công cụ giám sát toàn diện, phù hợp với đặc thù giao thông địa phương, tạo tiền đề cho các hệ thống cảnh báo thông minh hơn.

Thứ ba, tối ưu hóa quy trình triển khai và giảm thiểu chi phí phần cứng. Hướng tới việc phát triển các phiên bản hệ thống nhỏ gọn hơn (như sử dụng các module AI chuyên dụng hoặc hệ thống SoC tích hợp sẵn NPU), giúp giảm thiểu kích thước, điện năng tiêu thụ và chi phí sản xuất. Điều này sẽ giúp việc tích hợp hệ thống ADAS lên các dòng xe phổ thông trở nên dễ dàng và khả thi hơn về mặt kinh tế.

Cuối cùng, tích hợp sâu hơn vào hệ sinh thái phương tiện thông minh thông qua chuẩn kết nối V2X. Thay vì chỉ dựa vào cảm biến của riêng phương tiện, việc trao đổi thông tin với hạ tầng giao thông (đèn tín hiệu, camera đường phố) và các phương tiện xung quanh sẽ giúp hệ thống có được cái nhìn toàn cảnh (Global view), từ đó đưa ra các quyết định lái xe an toàn và hiệu quả hơn. Đây chính là mảnh ghép cuối cùng để biến các phương tiện cá nhân đơn lẻ thành một phần của mạng lưới giao thông vận tải thông minh, đồng bộ và bền vững.

Bảng 5.1: Tổng hợp và So sánh các phương pháp tiếp cận ADAS

Loại	Công nghệ tiêu biểu	Đặc điểm	Hạn chế
Thị giác máy tính	YOLOv11, OpenCV	Nắm bắt chi tiết làn đường, đối tượng thông qua hình ảnh.	Độ chính xác giảm khi thời tiết xấu hoặc thiếu sáng.
Hợp	Kalman, IMU,	Tăng độ chính	Yêu cầu sự đồng bộ

nhất cảm biến	CAN-Bus	xác vị trí, bù trừ nhiều động lực học.	hóa dữ liệu rất khắt khe.
Phân tích hành vi	LSTM, Deep Learning	Dự đoán rủi ro dựa trên chuỗi dữ liệu thời gian.	Cần tập dữ liệu huấn luyện lớn và thực tế.
Dẫn đường động	OSRM API, GPS	Hỗ trợ lộ trình thông minh, cảnh báo chệch hướng.	Phụ thuộc vào chất lượng sóng GPS và kết nối mạng.

TÀI LIỆU THAM KHẢO

- [1] J. Sobti et al., "Effect of Advanced Driver Assistance Systems (ADAS) on Pedestrian Safety," in *Proc. IEEE Conf.*, 2024.
- [2] L. Masello et al., "On the road safety benefits of advanced driver assistance systems in different driving contexts," *Transportation Research Interdisciplinary Perspectives*, vol. 15, 2022.
- [3] I. A. Ghany et al., "Implementation of YOLOv11 Algorithm on Raspberry Pi for Automated PCB Quality Inspection," in *Proc. IEEE Conf.*, 2024.
- [4] I. Suchý et al., "Review of large YOLOv11 and RT-DETR energy efficiency on heterogeneous edge platforms," *Scientific Reports*, 2026.
- [5] Ultralytics, "YOLOv11 Documentation," 2024.
- [6] J. Redmon et al., "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE CVPR*, 2016.
- [7] G. Welch and G. Bishop, "An Introduction to the Kalman Filter," Technical Report, *University of North Carolina at Chapel Hill*, 2006.
- [8] F. Şimşek et al., "Real-Time Object Detection with Quantized YOLOv11 on Raspberry Pi 5 for Low-Speed ADAS," 2026.
- [9] M. Kilicarslan and J. Y. Zheng, "Predict Vehicle Collision by TTC From Motion Using a Single Video Camera," *IEEE Trans. Intell. Transp. Syst.*, 2018.
- [10] Raspberry Pi Foundation, "Raspberry Pi 5 Product Brief," 2024.
- [11] Bosch, "CAN Specification Version 2.0," 1991.
- [12] Z. Qin et al., "Ultra Fast Structure-aware Deep Lane Detection," in *Proc. European Conference on Computer Vision (ECCV)*, 2020.

- [13] K. Wilke et al., "CAN: Controller Area Network: Basics, Protocols, Chips and Applications," *Springer Science & Business Media*, 2012.
- [14] H. Hattori et al., "Auto-calibration of camera-based ADAS using vanishing points," in *Proc. IEEE Intelligent Vehicles Symposium (IV)*, 2023.
- [15] SAE International, "Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles," *J3016_202104*, 2021.